PA-Q1-Part I

April 15, 2026

1 *PA - Part I: Basic Vision Models [Experiments with MNIST]* (55pt)

Keywords: Multiclass Image Classification, Neural Networks, PyTorch

MNIST * The MNIST database (Modified National Institute of Standards and Technology database) is a large database of handwritten digits that is commonly used for training various image processing systems. * The MNIST database contains 70,000 labeled images. Each datapoint is a 28×28 pixels grayscale image. * However to speed up computations, we will use a much smaller dataset with size 8×8 images. These images are loaded from `sklearn.datasets`.

Agenda: * The PA is split into three parts, the first part dealing with miniature models which we will build from scratch, the second part dealing with modern architectures and the bonus third part dealing with training vision models robust to adversarial attacks. * In this part, we will be performing multiclass classification on the simplified MNIST dataset from scratch. * We will be applying and analysing different loss functions, optimization techniques and learning methods.

- We will be using PyTorch to do most of the heavylifting for modeling and training.

Note: * Hardware acceleration (GPU) is recommended but not required for this part. * A note on working with GPU: * Take care that whenever declaring new tensors, set `device=device` in parameters. * You can also move a declared torch tensor/model to device using `.to(device)`. * To move a torch model/tensor to cpu, use `.to('cpu')` * Keep in mind that all the tensors/model involved in a computation have to be on the same device (CPU/GPU). * Run all the cells in order. * Only **add your code** to cells marked with "TODO:" or with "..." * You should not have to change variable names where provided, but you are free to if required for your implementation.

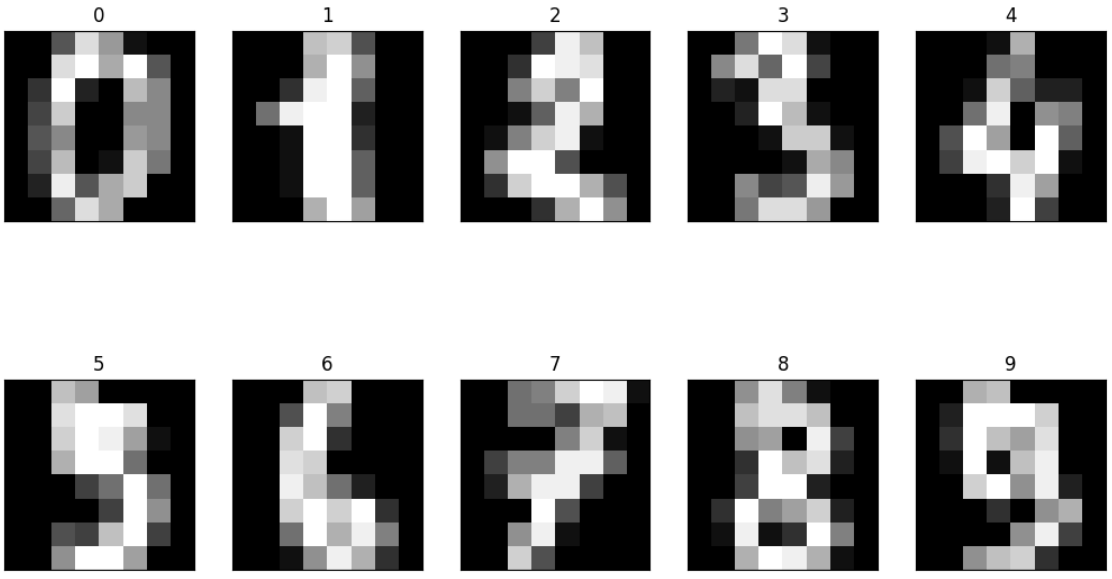
1.0.1 *Setup: Imports and Utils*

[1]:
```
# imports
import torch
import numpy as np
import math
from tqdm.notebook import tqdm
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
# loading the dataset directly from the scikit-learn library
dataset = load_digits()
X = dataset.data
y = dataset.target
print('Number of images:', X.shape[0])
print('Number of features per image:', X.shape[1])
```

```
Number of images: 1797
Number of features per image: 64
```

[2]:
```
# utility function to plot gallery of images
def plot_gallery(images, titles, height, width, n_row=2, n_col=4):
    plt.figure(figsize=(2* n_col, 3 * n_row))
    plt.subplots_adjust(bottom=0, left=0.01, right=0.99, top=0.90, hspace=0.35)
    for i in range(n_row * n_col):
        plt.subplot(n_row, n_col, i + 1)
        plt.imshow(images[i].reshape((height, width)), cmap=plt.cm.gray)
        plt.title(titles[i], size=12)
        plt.xticks(())
        plt.yticks(())

# visualize some of the images of the MNIST dataset
plot_gallery(X, y, 8, 8, 2, 5)
```



[3]:
```
# Let us split the dataset into training and test sets in a stratified manner.
# Note that we are not creating evaluation datset as we will not be tuning␣
 ↪hyper-parameters
```

```
# The split ratio is 4:1
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42, stratify=y)
print('Shape of train dataset:', X_train.shape)
print('Shape of evaluation dataset:', X_test.shape)
```

```
Shape of train dataset: (1437, 64)
Shape of evaluation dataset: (360, 64)
```

[4]:
```
# define some constants - useful for later
num_classes = len(np.unique(y)) # number of target classes = 10 --␣
 ↪(0,1,2,3,4,5,6,7,8,9)
num_features = X.shape[1]       # number of features = 64
max_epochs = 100000             # max number of epochs for training
lr = 1e-2                       # learning rate
tolerance = 1e-6                # tolerance for early stopping during training
```

[5]:
```
# Hardware Acceleration: to set device if using GPU.
# You can change runtime in colab by naviagting to (Runtime->Change runtime␣
 ↪type), and selecting GPU in hardware accelarator.
# NOTE that you can run this homework without GPU.
device = 'cuda' if torch.cuda.is_available() else 'cpu'
device
```

[5]:
```
'cuda'
```

1.0.2 *(a) Utils and pre-processing* (5pt)

- Scale the image data between 0 and 1
- One-hot encode the target data
- Convert the data to tensors and move them to the required device

[ ]:
```
# 1. Scale the features between 0 and 1
# To scale, you can directly use the MinMaxScaler from sklearn.
#######

from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# output variable names -  X_train, X_test
#######
```

[8]:
```
# 2. One-Hot encode the target labels
# To one-hot encode, you can use the OneHotEncoder from sklearn
#######

from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder(sparse_output=False)
y_train_ohe = ohe.fit_transform(y_train.reshape(-1, 1))
y_test_ohe = ohe.transform(y_test.reshape(-1, 1))

# output variable names -  y_train_ohe, y_test_ohe
#######
print('Shape of y_train_ohe:',y_train_ohe.shape)
print('Shape of y_test_ohe:',y_test_ohe.shape)


# move X and y to the defined device and convert them to torch.float32
X_train_torch = torch.tensor(X_train, dtype=torch.float32, device=device)
X_test_torch = torch.tensor(X_test, dtype=torch.float32, device=device)
y_train_ohe_torch = torch.tensor(y_train_ohe, dtype=torch.float32,␣
 ↪device=device)
y_test_ohe_torch = torch.tensor(y_test_ohe, dtype=torch.float32, device=device)

# output variable names -  X_train_torch, X_test_torch, y_train_ohe_torch,␣
 ↪y_test_ohe_torch
```

```
Shape of y_train_ohe: (1437, 10)
Shape of y_test_ohe: (360, 10)
```

1.0.3 *(b) Implement Multi-Class Logistic Regression from scratch* (15pt)

In this problem, we will apply multiclass logistic regression from scratch trained using gradient descent (GD) with Mean Squared Error (MSE) loss as the objective.

We will be using a linear model

$$y^{(i)} = W\mathbf{x}^{(i)},$$

with the following notations:

- Weight matrix:

$$W_{p\times n} = \begin{bmatrix} \leftarrow & \mathbf{w}_1^\top & \rightarrow \\ \leftarrow & \mathbf{w}_2^\top & \rightarrow \\ & \vdots & \\ \leftarrow & \mathbf{w}_p^\top & \rightarrow \end{bmatrix},$$

where (p) is the number of target classes.

- Data points:

$$\mathbf{x}^{(i)} \in \mathbb{R}^n, \qquad y^{(i)} \in \mathbb{R},$$

and

$$X = \begin{bmatrix} \uparrow & \uparrow & \cdots & \uparrow \\ \mathbf{x}^{(1)} & \mathbf{x}^{(2)} & \cdots & \mathbf{x}^{(m)} \\ \downarrow & \downarrow & \cdots & \downarrow \end{bmatrix}, \qquad Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix},$$

where (m) is the number of datapoints.

Note: Here we need to define the model prediction. The input matrix is $X_{n \times m}$ where m is the number of examples, and n is the number of features. The linear predictions can be given by: $Y = WX + b$ where W is a $p \times n$ weight matrix and b is a p size bias vector. p is the number of target classes.

#1. Define a function `linear_model`

- This function takes as input a weight matrix (`W`), bias vector (`b`), and input data matrix of size $m \times n$ (`XT`).
- This function should return the predictions $\hat{y}$, which is an $m \times p$ matrix. For each datapoint row which is a $1 \times p$ vector, the prediction can be seen as the scores that the model gives each target class for that datapoint

[9]:
```
#######
def linear_model(W, b, XT):
    y_hat = torch.mm(XT, W) + b
    return y_hat

#######
```

Note: The loss function that we would be using is the Mean Square Error (L2) Loss: $MSE = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2$, where m is the number of examples, $\hat{y}^{(i)}$ is the predicted value of $x^{(i)}$ and $y^{(i)}$ is the ground truth of $x^{(i)}$.

#2. Define a function `mse_loss`

- This function takes as input prediction (`y_pred`) and ground-truth label (`y`), and returns the MSE loss.

[10]:
```
#######
def mse_loss(y_pred, y):
    loss = torch.mean((y_pred - y) ** 2)
    return loss

#######
```

#3. Define a function: `initializeWeightsAndBiases` In this part, we will do some setup required for training (such as initializing weights and biases) and move everything to torch tensors.

- This function returns tuple `(W, b)`, where `W` is a randomly generated torch tensor of size `num_classes x num_features`, and `b` is a randomly generated torch vector of size `num_classes`.
- For both the tensors, set `requires_grad=True` in parameters.

[11]:
```
#######
def initializeWeightsAndBiases(num_classes, num_features):
    W = torch.rand(num_features, num_classes, device=device, dtype=torch.
 ↪float32, requires_grad=True)
    b = torch.rand(num_classes, device=device, dtype=torch.float32,␣
 ↪requires_grad=True)
    return W, b
#######
```

#4 Training code

- Given below is a function: `train_linear_regression_model` that takes as inputs as max number of epochs (`max_epochs`), weights (`W`), biases (`b`), training data (`X_train, y_train`), learning rate (`lr`), tolerance for stopping (`tolerance`).
- This function returns a tuple `(W,b,losses)` where `W,b` are the trained weights and biases respectively, and `losses` is a list of tuples of loss logged every 100^{th} epoch.
- You can go through this article for reference.

[12]:
```
# Define a function train_linear_regression_model
def train_linear_regression_model(max_epochs, W, b, X_train, y_train, lr,␣
 ↪tolerance):
  losses = []
  prev_loss = float('inf')

  for epoch in tqdm(range(max_epochs)):

      #######
      # 7. do prediction
      y_pred = linear_model(W, b, X_train)

      # 8. get the loss
      loss = mse_loss(y_pred, y_train)

      # 9. backpropagate loss
      loss.backward()

      # 10. update the weights and biasees
      with torch.no_grad():
          W -= lr * W.grad
          b -= lr * b.grad
```

```
    # 11. set the gradients to zero
    W.grad.zero_()
    b.grad.zero_()

    #######

    # log the loss every 100th epoch and print every 5000th epoch:
    if epoch%100==0:
      losses.append((epoch, loss.item()))
      if epoch%5000==0:
        print('Epoch: {}, Loss: {}'.format(epoch, loss.item()))

    # break if decrease in loss is less than threshold
    if prev_loss - loss.item() < tolerance:
      print(f'Early stopping at epoch {epoch}')
      break
    prev_loss = loss.item()

  # return updated weights, biases, and logged losses
  return W, b, losses
```

#5. Initialize parameters and train your own model

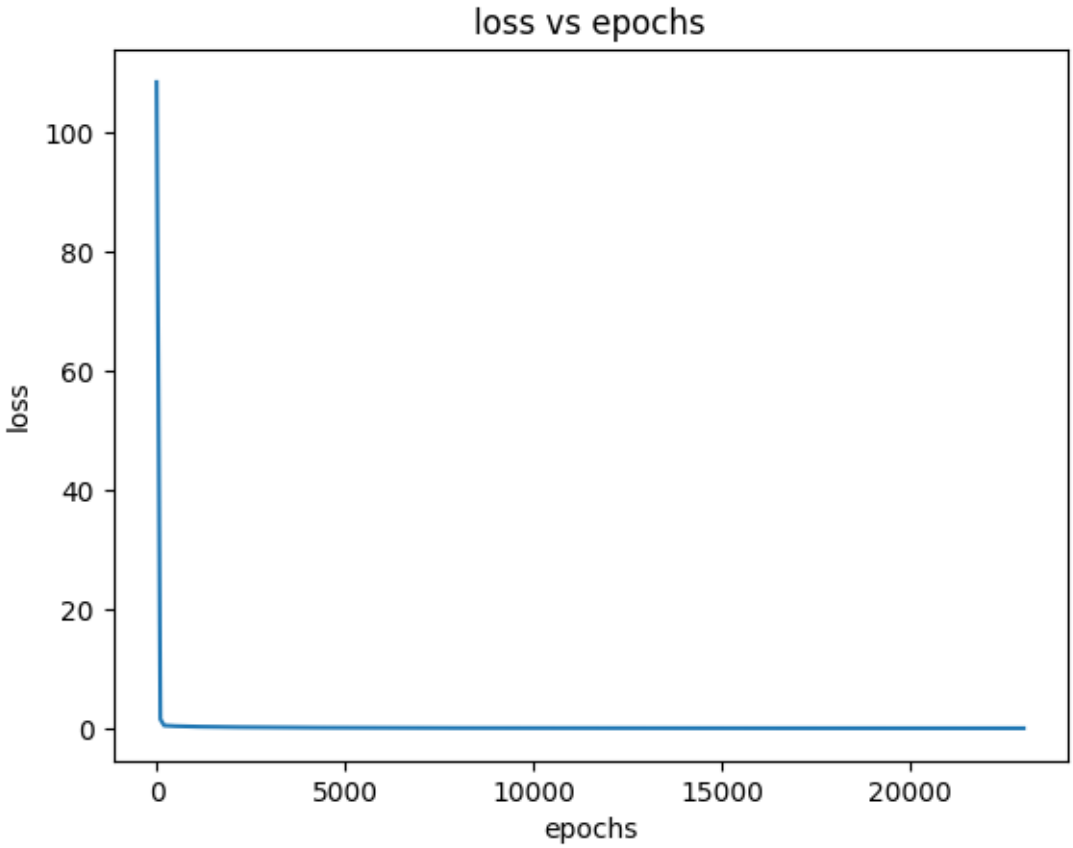
- Initialize weights and biases using the `initializeWeightsAndBiases` function that you defined earlier
- Train your model using function `train_linear_regression_model` defined above.
- Use full batch (set `batch_size=len(X_train)` for training (Gradient Descent).
- Also plot the graph of loss vs number of epochs (Recall that values for learning rate (`lr`) and tolerance (`tolerance`) are already defined above).

[14]:
```
W, b = initializeWeightsAndBiases(num_classes, num_features)
```

[16]:
```
#######
W, b = initializeWeightsAndBiases(num_classes, num_features)
W, b, losses = train_linear_regression_model(max_epochs, W, b, X_train_torch,␣
 ↪y_train_ohe_torch, lr, tolerance)

import matplotlib.pyplot as plt
plt.plot([x[0] for x in losses],[x[1] for x in losses])
plt.xlabel('epochs')
plt.ylabel('loss')
plt.title('loss vs epochs')
plt.show()

#######
```

```
  0%|          | 0/100000 [00:00<?, ?it/s]
```

```
Epoch: 0, Loss: 108.47657775878906
Epoch: 5000, Loss: 0.13736692070960999
Epoch: 10000, Loss: 0.08372224122285843
Epoch: 15000, Loss: 0.06497304141521454
Epoch: 20000, Loss: 0.05588589236140251
Early stopping at epoch 23048
```



[18]:
```
# print accuracies of model
predictions_train = linear_model(W, b, X_train_torch)
predictions_test = linear_model(W, b, X_test_torch)
y_train_pred = torch.argmax(predictions_train, dim=1).cpu().numpy()
y_test_pred = torch.argmax(predictions_test, dim=1).cpu().numpy()
print("Train accuracy:",accuracy_score(y_train_pred, np.asarray(y_train,␣
 ↪dtype=np.float32)))
print("Test accuracy:",accuracy_score(y_test_pred, np.asarray(y_test, dtype=np.
 ↪float32)))
```

```
Train accuracy: 0.9137091162143354
Test accuracy: 0.9305555555555556
```

1.0.4 *(c) Use PyTorch for training* (10pt)

- In the previous part, we defined the model, loss, and even the gradient update step. We also had to manually set the gradients to zero.
- In this part, we will re-implement the linear model and see how we can directly use Pytorch to do all this for us in a few simple steps.

[24]:
```
# common utility function to print accuracies
def print_accuracies_torch(model, X_train_torch, X_test_torch, y_train, y_test):
  predictions_train = model(X_train_torch)
  predictions_test = model(X_test_torch)
  y_train_pred = torch.argmax(predictions_train, dim=1).cpu().numpy()
  y_test_pred = torch.argmax(predictions_test, dim=1).cpu().numpy()
  print("Train accuracy:",accuracy_score(y_train_pred, np.asarray(y_train,␣
 ↪dtype=np.float32)))
  print("Test accuracy:",accuracy_score(y_test_pred, np.asarray(y_test,␣
 ↪dtype=np.float32)))
```

#1. Define the linear model using PyTorch

- Use the inbuilt PyTorch `torch.nn.Module` to define a model class.
- Use the `torch.nn.Linear` to define the linear layer of the model

[20]:
```
#######
# Define a model class using torch.nn
class Linear_Model(torch.nn.Module):
  def __init__(self):
    super(Linear_Model, self).__init__()
    # Initalize various layers of model as below
    # 1. initialze one linear layer: num_features -> num_targets
    self.linear = torch.nn.Linear(num_features, num_classes)

  def forward(self, X):
    # 2. define the feedforward algorithm of the model and return the final␣
 ↪output
    x = self.linear(X)
    return x

#######
```

#2. A general function for training a PyTorch model. Define a general training function: `train_torch_model`. * This function takes as input an initialized torch model (`model`), batch size (`batch_size`), initialized loss (`criterion`), max number of epochs (`max_epochs`), training data (`X_train, y_train`), learning rate (`lr`), tolerance for stopping (`tolerance`). * This function will return a tuple `(model, losses)`, where `model` is the trained model, and `losses` is a list of tuples of loss logged every 100^{th} epoch.

Note: You can reuse a lot of components from the `train_linear_regression_model` function from the previous section

You can go through this article for reference.

[21]:
```
# Define a function train_torch_model
def train_torch_model(model, batch_size, criterion, max_epochs, X_train,␣
 ↪y_train, lr, tolerance):
  losses = []
  prev_loss = float('inf')
  number_of_batches = math.ceil(len(X_train)/batch_size)

  #######
  # 3. move model to device
  model.to(device)

  # 4. define optimizer (use torch.optim.SGD (Stochastic Gradient Descent))
  # Set learning rate to lr and also set model parameters
  optimizer =  torch.optim.SGD(model.parameters(), lr=lr)

  for epoch in tqdm(range(max_epochs)):
    for i in range(number_of_batches):
      X_train_batch = X_train[i*batch_size:(i+1)*batch_size].to(device)
      y_train_batch = y_train[i*batch_size:(i+1)*batch_size].to(device)

      # 5. reset gradients
      optimizer.zero_grad()

      # 6. prediction
      prediction = model(X_train_batch)

      # 7. calculate loss
      loss = criterion(prediction, y_train_batch)

      # 8. backpropagate loss
      loss.backward()

      # 9. perform a single gradient update step
      optimizer.step()
      optimizer.zero_grad()

  #######

    # log loss every 100th epoch and print every 5000th epoch:
    if epoch%100==0:
      losses.append((epoch, loss.item()))
      if epoch%5000==0:
        print('Epoch: {}, Loss: {}'.format(epoch, loss.item()))

    # break if decrease in loss is less than threshold
```

```
    if prev_loss - loss.item() < tolerance:
      print(f'Early stopping at epoch {epoch}')
      break
    prev_loss = loss.item()

  # return updated model and logged losses
  return model, losses
```

1.0.5 *(d) Working with different model types* (15pt)

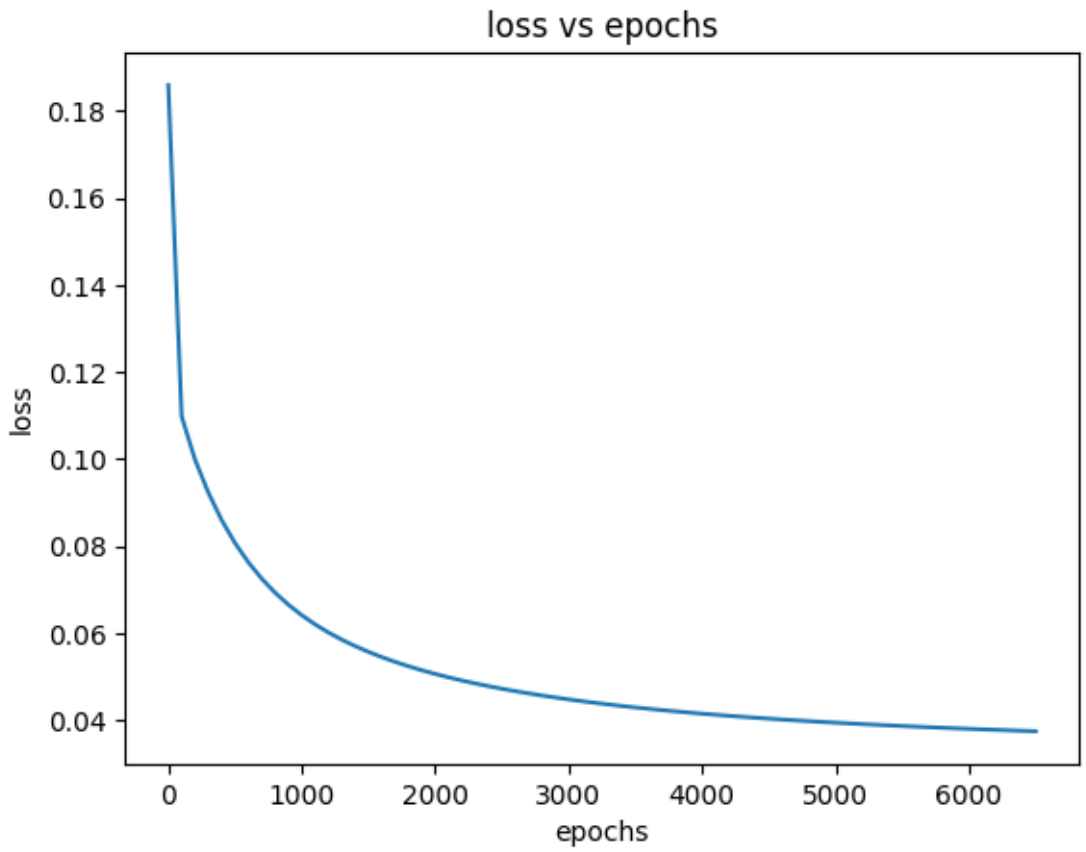
- Now, we retrain the above model with `batch_size=64`
- Use Stochastic/Mini-batch Gradient Descent and keep everything else the same.
- Like before, we plot the graph between loss and number of epochs.

#1. MSE Loss and Gradient Descent (GD)

- Use `nn.MSELoss`.
- Use full batch for training (Gradient Descent).
- Also plot the graph of loss vs number of epochs.

[26]:
```
#######
model = Linear_Model()
criterion = torch.nn.MSELoss()
batch_size = len(X_train_torch)
model, losses = train_torch_model(model, batch_size, criterion, max_epochs,␣
 ↪X_train_torch, y_train_ohe_torch, lr, tolerance)

import matplotlib.pyplot as plt
plt.plot([x[0] for x in losses],[x[1] for x in losses])
plt.xlabel('epochs')
plt.ylabel('loss')
plt.title('loss vs epochs')
plt.show()
#######
```

```
  0%|          | 0/100000 [00:00<?, ?it/s]
Epoch: 0, Loss: 0.18589311838150024
Epoch: 5000, Loss: 0.039513543248176575
Early stopping at epoch 6554
```



[27]:
```
# print accuracies of model
print_accuracies_torch(model, X_train_torch, X_test_torch, y_train, y_test)
```

```
Train accuracy: 0.9290187891440501
Test accuracy: 0.9111111111111111
```

#2. MSE Loss and Stochastic Gradient Descent (SGD)

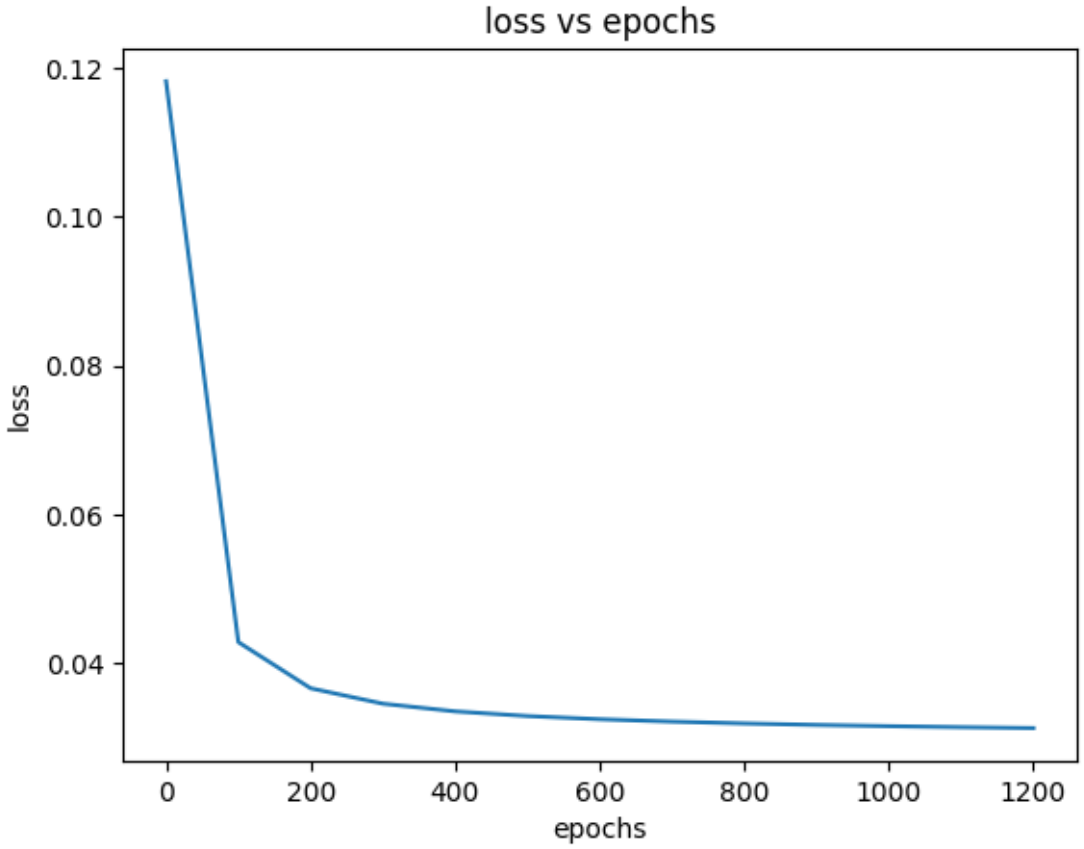
- Use `nn.MSELoss`.
- Use `batch_size=64` for training
- Also plot the graph of loss vs number of epochs.

[28]:
```
#######
model = Linear_Model()
criterion = torch.nn.MSELoss()
batch_size = 64
model, losses = train_torch_model(model, batch_size, criterion, max_epochs,␣
 ↪X_train_torch, y_train_ohe_torch, lr, tolerance)

import matplotlib.pyplot as plt
```

```
plt.plot([x[0] for x in losses],[x[1] for x in losses])
plt.xlabel('epochs')
plt.ylabel('loss')
plt.title('loss vs epochs')
plt.show()
#######
```

```
  0%|          | 0/100000 [00:00<?, ?it/s]
```

```
Epoch: 0, Loss: 0.11823853105306625
Early stopping at epoch 1275
```



[29]:
```
# print accuracies of model
print_accuracies_torch(model, X_train_torch, X_test_torch, y_train, y_test)
```

```
Train accuracy: 0.9457202505219207
Test accuracy: 0.9472222222222222
```

Cross-Entropy (CE) Loss with Linear Model

- Instead of using MSE Loss, we will use a much more natural loss function for the multi-class logistic regression task which is the Cross Entropy Loss.
- CE loss converts the model scores of each class into a probability.
- This model penalizes both choosing the wrong class as well as uncertainty (choosing the right class with low probability).
- And we will use the same linear model defined in **(c)**.

Note: The Cross Entropy Loss for multiclass classification is the mean of the negative log likelihood of the output logits after softmax:

$$L = \frac{1}{m}\sum_{i=1}^{m} \underbrace{\underbrace{\underbrace{-\,y^{(i)}\, log \underbrace{\frac{e^{\hat{y}^{(i)}}}{\sum_{j=1}^{p} e^{\hat{y}^{(j)}}}}_{\text{Softmax}}}_{\text{LogSoftmax}}}_{\text{Negative Log Likelihood (NLL)}}}_{\text{Cross Entropy (CE) Loss}} ,$$

where $y^{(i)}$ is the ground truth, and $\hat{y}^{(k)}$ (also called as *logits*) represent the outputs of the last linear layer of the model.

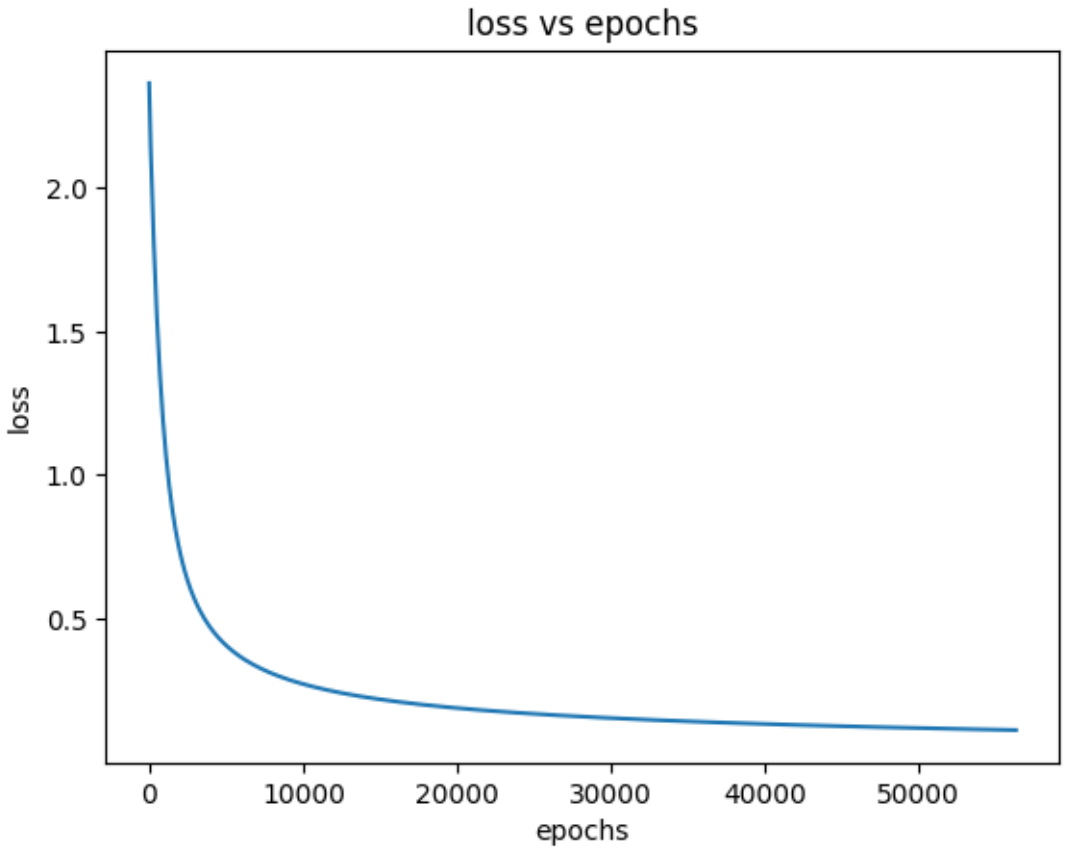
#3. CE Loss and GD

- Instead of `nn.MSELoss`, train the linear model with `nn.CrossEntropyLoss`.
- Use **full-batch**.
- Also plot the graph between loss and number of epochs.

[30]:
```
#######
model = Linear_Model()
criterion = torch.nn.CrossEntropyLoss()
batch_size = len(X_train_torch)
model, losses = train_torch_model(model, batch_size, criterion, max_epochs,␣
 ↪X_train_torch, y_train_ohe_torch, lr, tolerance)

import matplotlib.pyplot as plt
plt.plot([x[0] for x in losses],[x[1] for x in losses])
plt.xlabel('epochs')
plt.ylabel('loss')
plt.title('loss vs epochs')
plt.show()
#######
```

```
  0%|          | 0/100000 [00:00<?, ?it/s]
Epoch: 0, Loss: 2.362581253051758
Epoch: 5000, Loss: 0.40804776549339294
Epoch: 10000, Loss: 0.2736990749835968
Epoch: 15000, Loss: 0.220407634973526
Epoch: 20000, Loss: 0.1901596486568451
Epoch: 25000, Loss: 0.17001718282699585
Epoch: 30000, Loss: 0.15531741082668304
Epoch: 35000, Loss: 0.14393536746501923
```

```
Epoch: 40000, Loss: 0.13475114107131958
Epoch: 45000, Loss: 0.12711308896541595
Epoch: 50000, Loss: 0.12061270326375961
Epoch: 55000, Loss: 0.11497961729764938
Early stopping at epoch 56363
```



[31]:
```
# print accuracies of model
print_accuracies_torch(model, X_train_torch, X_test_torch, y_train, y_test)
```

```
Train accuracy: 0.9812108559498957
Test accuracy: 0.9638888888888889
```

#4. CE Loss and SGD

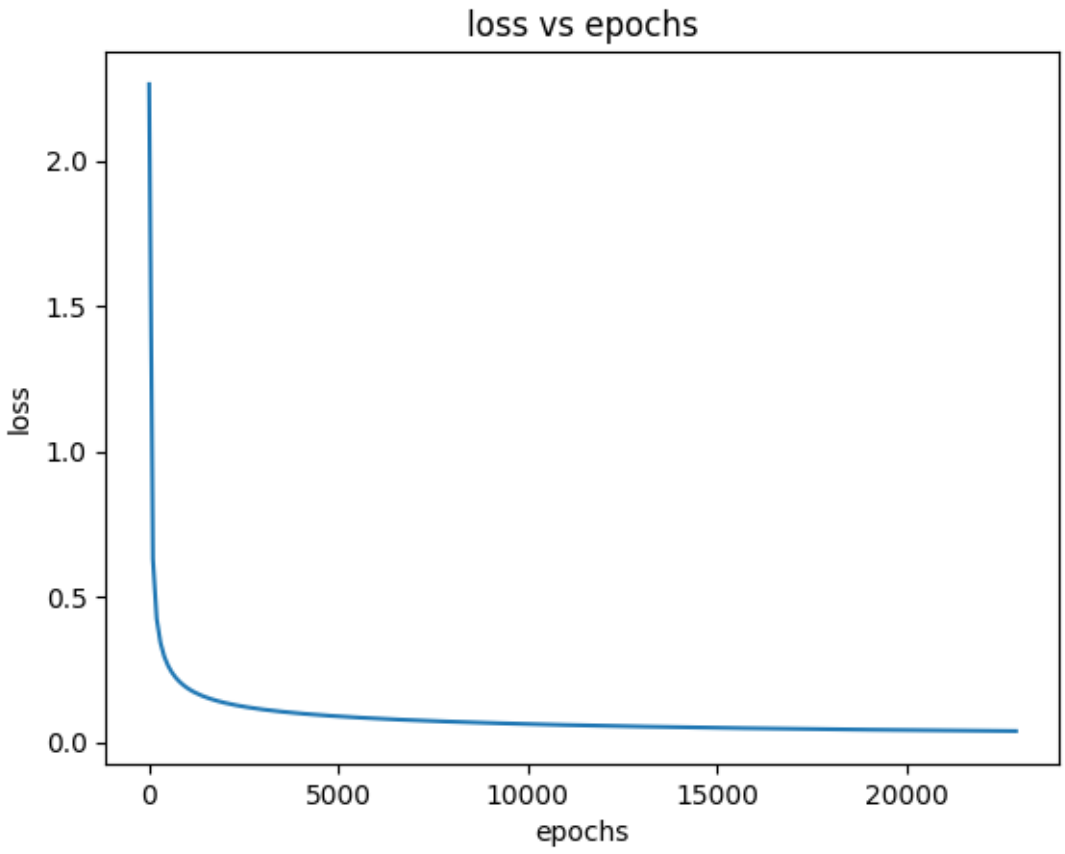
- Use a different batch size, `batch_size=64` with the new loss and repeat the previous part #1.
- Also plot the graph of loss vs epochs.

[32]:
```
#######
model = Linear_Model()
criterion = torch.nn.CrossEntropyLoss()
```

```
batch_size = 64
model, losses = train_torch_model(model, batch_size, criterion, max_epochs,␣
 ↪X_train_torch, y_train_ohe_torch, lr, tolerance)

import matplotlib.pyplot as plt
plt.plot([x[0] for x in losses],[x[1] for x in losses])
plt.xlabel('epochs')
plt.ylabel('loss')
plt.title('loss vs epochs')
plt.show()
#######
```

```
  0%|          | 0/100000 [00:00<?, ?it/s]
```

```
Epoch: 0, Loss: 2.2636024951934814
Epoch: 5000, Loss: 0.0880981907248497
Epoch: 10000, Loss: 0.06155843660235405
Epoch: 15000, Loss: 0.04867053031921387
Epoch: 20000, Loss: 0.04074279963970184
Early stopping at epoch 22963
```



[33]:
```
# print accuracies of model
print_accuracies_torch(model, X_train_torch, X_test_torch, y_train, y_test)
```

```
Train accuracy: 0.9979123173277662
Test accuracy: 0.9694444444444444
```

Training a neural network model in PyTorch

- We will train a neural network in pytorch with two hidden layers of sizes 32 and 16 neurons.
- We will use non-linear ReLU activations to effectively make it a non-linear model.
- We will use this neural network model for multi-class classification with Cross Entropy Loss.

Note: The neural network model output can be represented mathematically as below: $\hat{y}^{(i)}_{10\times 1} = W^{(3)}_{10\times 16}\sigma(W^{(2)}_{16\times 32}\sigma(W^{(1)}_{32\times 64}\mathbf{x}^{(i)}_{64\times 1} + \mathbf{b}^{(1)}_{32\times 1}) + \mathbf{b}^{(2)}_{16\times 1}) + \mathbf{b}^{(3)}_{10\times 1}$, where σ represents ReLU activation, $W^{(i)}$ is the weight of the i^{th} linear layer, and $\mathbf{b}^{(i)}$ is the layer's bias. We use the subscript to denote the dimension for clarity.

#5. Define the 2 hidden-layer Neural Network (NN)

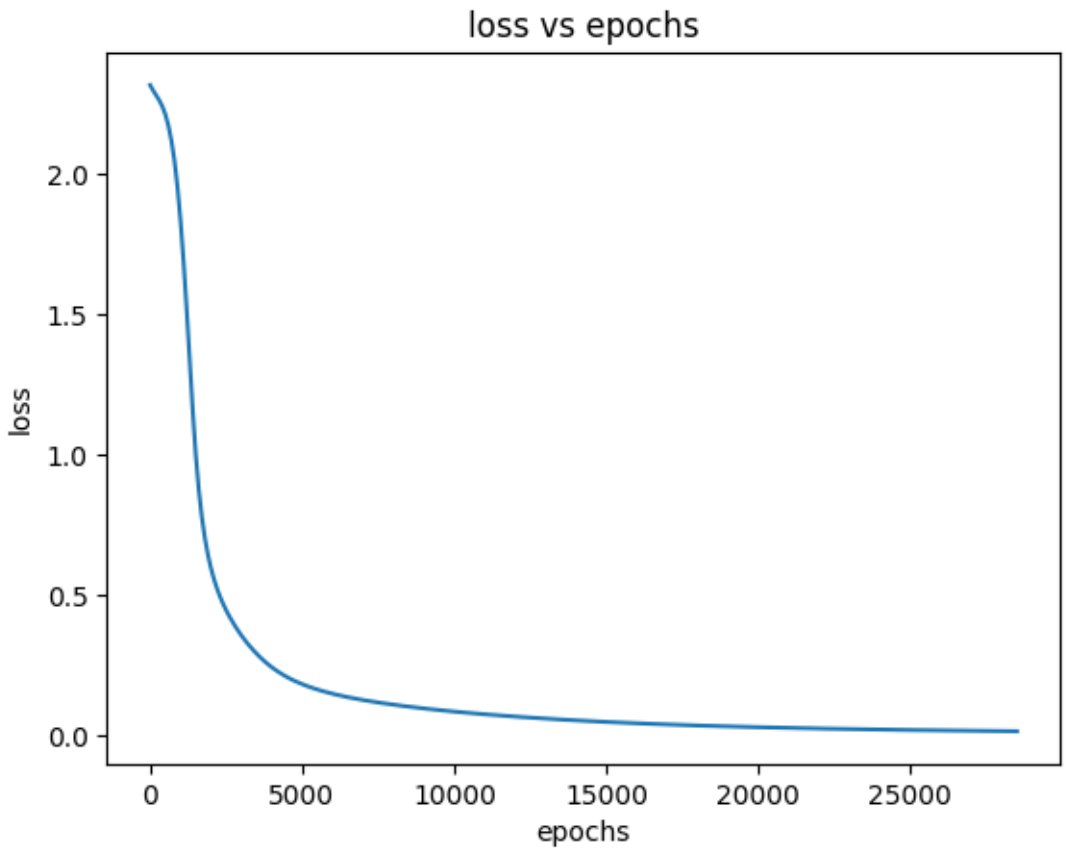
[34]:
```
#######
# Define a neural network model class using torch.nn
class NN_Model(torch.nn.Module):
  def __init__(self):
    super(NN_Model, self).__init__()
    # Initalize various layers of model as instructed below
    # 1. initialize three linear layers: num_features -> 32, 32 -> 16, 16 ->␣
 ↪num_targets
    self.linear1 = torch.nn.Linear(num_features, 32)
    self.linear2 = torch.nn.Linear(32, 16)
    self.linear3 = torch.nn.Linear(16, num_classes)

    # 2. initialize RELU
    self.relu = torch.nn.ReLU()

  def forward(self, X):
    # 3. define the feedforward algorithm of the model and return the final␣
 ↪output
    # Apply non-linear ReLU activation between subsequent layers
    x = self.linear1(X)
    x = self.relu(x)
    x = self.linear2(x)
    x = self.relu(x)
    x = self.linear3(x)
    return x

#######
```

#6. NN with CE Loss and GD

- Use Cross Entropy Loss.
- Use full-batch and plot the graph of loss vs number of epochs.
- Note that you can re-use the training function `train_torch_model` (from part (b)).

[35]:
```
#######
model = NN_Model()
criterion = torch.nn.CrossEntropyLoss()
batch_size = len(X_train_torch)
model, losses = train_torch_model(model, batch_size, criterion, max_epochs,␣
 ↪X_train_torch, y_train_ohe_torch, lr, tolerance)

import matplotlib.pyplot as plt
plt.plot([x[0] for x in losses],[x[1] for x in losses])
plt.xlabel('epochs')
plt.ylabel('loss')
plt.title('loss vs epochs')
plt.show()
#######
```

```
  0%|          | 0/100000 [00:00<?, ?it/s]
Epoch: 0, Loss: 2.3167550563812256
Epoch: 5000, Loss: 0.1832629293203354
Epoch: 10000, Loss: 0.08558323234319687
Epoch: 15000, Loss: 0.048801910132169724
Epoch: 20000, Loss: 0.030072282999753952
Epoch: 25000, Loss: 0.02008567377924919
Early stopping at epoch 28593
```



[36]:
```
# print accuracies of model
print_accuracies_torch(model, X_train_torch, X_test_torch, y_train, y_test)
```

```
Train accuracy: 0.9993041057759221
Test accuracy: 0.9638888888888889
```

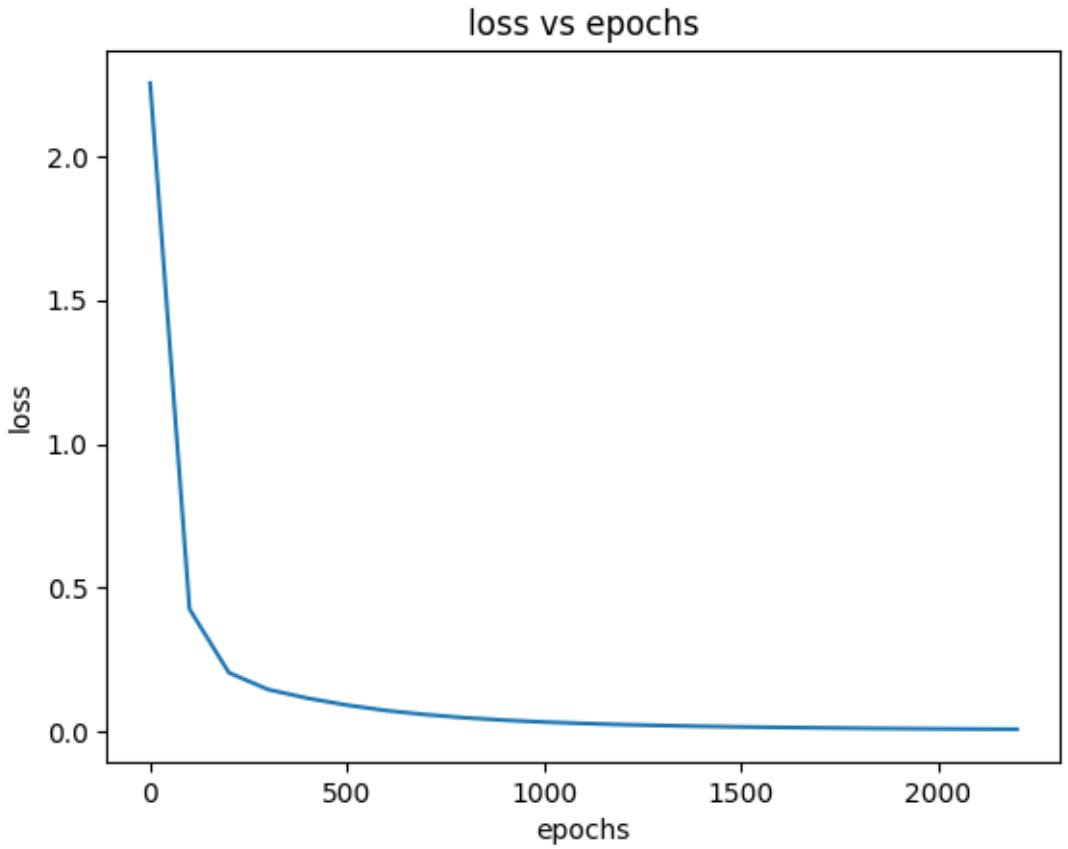
#7. NN with CE Loss and SGD

- Re-train the above model with `batch_size=64`.
- Also plot the graph of loss vs epochs.

[37]:
```
#######
model = NN_Model()
criterion = torch.nn.CrossEntropyLoss()
batch_size = 64
model, losses = train_torch_model(model, batch_size, criterion, max_epochs,␣
 ↪X_train_torch, y_train_ohe_torch, lr, tolerance)

import matplotlib.pyplot as plt
plt.plot([x[0] for x in losses],[x[1] for x in losses])
```

```
plt.xlabel('epochs')
plt.ylabel('loss')
plt.title('loss vs epochs')
plt.show()
#######
```

```
  0%|          | 0/100000 [00:00<?, ?it/s]
Epoch: 0, Loss: 2.254425287246704
Early stopping at epoch 2237
```



[38]:
```
# print accuracies of model
print_accuracies_torch(model, X_train_torch, X_test_torch, y_train, y_test)
```

```
Train accuracy: 1.0
Test accuracy: 0.9805555555555555
```

1.0.6 *(e) Analyze the results* (10pt)

In the above few examples, we performed several experiments with different batch size and loss functions. Now it's time to analyze our observations from the results.

Recall that we trained the following models in this tutorial:

1. Linear Model - Scratch + MSE Loss + Full Batch (GD)
2. Linear Model - PyTorch + MSE Loss + Full Batch
3. Linear Model - PyTorch + MSE Loss + Mini Batch (SGD)
4. Linear Model - PyTorch + CE Loss + Full Batch
5. Linear Model - PyTorch + CE Loss + Mini Batch
6. NN Model - PyTorch + CE Loss + Full Batch
7. NN Model - PyTorch + CE Loss + Mini Batch

[39]:
```
# plotting graph of accuracy vs model
import pandas as pd
import plotly.express as px
xlabels = ['linear-scratch', 'linear-torch-full', 'linear-torch-batch',␣
 ↪'linear-celoss-full', 'linear-celoss-batch', 'nn-full', 'nn-batch']
train_accuracies = [0.9137091162143354, 0.9290187891440501, 0.9457202505219207,␣
 ↪0.9812108559498957, 0.9979123173277662, 0.9993041057759221, 1.0]
test_accuracies = [0.9305555555555556, 0.9111111111111111, 0.9472222222222222,␣
 ↪0.9638888888888889, 0.9694444444444444, 0.9638888888888889, 0.
 ↪9805555555555555]

tuples = [(xlabels[i],train_accuracies[i], 'train' ) for i in␣
 ↪range(len(xlabels))]+[(xlabels[i],test_accuracies[i], 'test' ) for i in␣
 ↪range(len(xlabels))]
df = pd.DataFrame(tuples, columns = ['model', 'accuracy', 'type'])
px.line(df, x='model', y='accuracy', color = 'type', markers=True, title =␣
 ↪'accuracy vs. model')
```

#1.Analysis Effect of using full vs. batch gradient descent:
Batch GD always outperformed its full batch counterpart in terms of accuracy. Batch GD also reached a better accuracy faster than its full batch counterpart. This is expected as full batch (GD) uses the entire training dataset to compute the gradient at each step, leading to a stable loss curve but extremely slow updates. On the otherhand, batched GD (SGD) has noisier gradients as there are significatly more updates in batches per epoch.

Effect of using different loss strategy:
MSE treats the problem like linear regression, which doesn't exactly align well for a classification problem like MNIST. Consequently, we do see that all CE loss models perform better than the MSE models with higher training and testing accuracies.

Effect of using linear vs. non-linear models:
Arguably MNIST is not linearly separable however the CE loss models performed decently. Still, the NN models performed better as these models can learn complex patterns related to each digit.

Training time per epoch in different cases:

Batch GD was always slower compared to full batch, as I was using GPU. With a dataset size of 1437 and batch size of 64, we are running about 22x more computations than the full batch, which ultimately is minimal in terms of GPU usage. Consequently the overhead of the 22 computations is significant compared to the single full batch computation. This is the case for both total runtime and training time per epoch. The linear models were the fastest compared to the NN models, which is also expected due to the additional multiple linear + ReLu layer computations.

PA-Q1-Part II

April 15, 2026

1 *PA - Part II: Advanced Vision Models [Experiments with CIFAR10]* (45pt)

Keywords: Multiclass Image Classification, ResNet, Vision Transformers, CLIP

CIFAR10 * The CIFAR10 database is a large database with images of objects like airplane, bird, cat, dog, etc. * It contains 70,000 labeled images. Each datapoint is a 32×32 pixels RGB image. * To provide a realistic problem, we will use the images at this standard resolution of 32×32.

Agenda: * The PA is split into three parts, the first part dealing with miniature models which we will build from scratch, the second part dealing with modern architectures and the bonus third part dealing with training vision models robust to adversarial attacks. * In this part, we will be moving towards more modern architectures and a more realistic problem, finetuning pretained model of different architectures on the CIFAR10 dataset and comparing its performance with models trained from scratch (initialized with random weights). * We will also be evaluating CLIP, which uses an architecture which is a backbone or helper model in most modern LLMs for image analysis and even in image generation models. * We will be using PyTorch and HuggingFace for loading and reusing complex model architecture.

Note: * Hardware acceleration (GPU) is **highly recommended** for this part as we will be training comparitively larger models! * If you are running locally, increase the `num_workers` parameter in the dataloaders to speed up computations. As Colab vCPUs have limits, it tends to be slower. * A note on working with GPU: * Take care that whenever declaring new tensors, set `device=device` in parameters. * You can also move a declared torch tensor/model to device using `.to(device)`. * To move a torch model/tensor to cpu, use `.to('cpu')` * Keep in mind that all the tensors/model involved in a computation have to be on the same device (CPU/GPU). * Run all the cells in order. * Only **add your code** to cells marked with "TODO:" or with "…" * You should not have to change variable names where provided, but you are free to if required for your implementation.

1.0.1 Pre-training and fine-tuning

- In this part, we want to use modern architectures, namely the **ResNet** and **Vision Transformer**, for classification on the CIFAR10 dataset.
- Instead of trying to train these modern architectures from random weights for our task, we can instead use **pre-trained** models that have already been trained on a large amount of data.
- We can then **fine-tune** these models for our specific tasks.

- ImageNet is one such large-scale dataset (~1.3 million images) that is widely used for pre-training neural network models.
- These pretrained models serve as good initializations for various image-related tasks.
- We compare fine-tuned models and models trained from random initialization to analyze the difference in performance of these approaches.

1.0.2 *Setup: Imports and Utils*

[1]:
```
!pip install open_clip_torch
```

```
Collecting open_clip_torch
  Downloading open_clip_torch-3.3.0-py3-none-any.whl.metadata (32 kB)
Requirement already satisfied: torch>=2.0 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from open_clip_torch) (2.9.1+cu130)
Requirement already satisfied: torchvision in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from open_clip_torch) (0.24.1+cu130)
Requirement already satisfied: regex in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from open_clip_torch) (2026.2.28)
Collecting ftfy (from open_clip_torch)
  Downloading ftfy-6.3.1-py3-none-any.whl.metadata (7.3 kB)
Requirement already satisfied: tqdm in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from open_clip_torch) (4.67.3)
Requirement already satisfied: huggingface-hub in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from open_clip_torch) (1.5.0)
Requirement already satisfied: safetensors in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from open_clip_torch) (0.7.0)
Collecting timm>=1.0.17 (from open_clip_torch)
  Downloading timm-1.0.26-py3-none-any.whl.metadata (39 kB)
Requirement already satisfied: pyyaml in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from timm>=1.0.17->open_clip_torch) (6.0.3)
Requirement already satisfied: filelock in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from torch>=2.0->open_clip_torch) (3.25.0)
Requirement already satisfied: typing-extensions>=4.10.0 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from torch>=2.0->open_clip_torch) (4.15.0)
Requirement already satisfied: sympy>=1.13.3 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from torch>=2.0->open_clip_torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
```

```
(from torch>=2.0->open_clip_torch) (3.6.1)
Requirement already satisfied: jinja2 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from torch>=2.0->open_clip_torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from torch>=2.0->open_clip_torch) (2026.2.0)
Requirement already satisfied: setuptools in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from torch>=2.0->open_clip_torch) (82.0.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from sympy>=1.13.3->torch>=2.0->open_clip_torch) (1.3.0)
Requirement already satisfied: wcwidth in
c:\users\tangykiwi\appdata\roaming\python\python313\site-packages (from
ftfy->open_clip_torch) (0.2.14)
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from huggingface-hub->open_clip_torch) (1.3.2)
Requirement already satisfied: httpx<1,>=0.23.0 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from huggingface-hub->open_clip_torch) (0.28.1)
Requirement already satisfied: packaging>=20.9 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from huggingface-hub->open_clip_torch) (26.0)
Requirement already satisfied: typer in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from huggingface-hub->open_clip_torch) (0.24.1)
Requirement already satisfied: anyio in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from httpx<1,>=0.23.0->huggingface-hub->open_clip_torch) (4.12.1)
Requirement already satisfied: certifi in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from httpx<1,>=0.23.0->huggingface-hub->open_clip_torch) (2026.2.25)
Requirement already satisfied: httpcore==1.* in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from httpx<1,>=0.23.0->huggingface-hub->open_clip_torch) (1.0.9)
Requirement already satisfied: idna in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from httpx<1,>=0.23.0->huggingface-hub->open_clip_torch) (3.11)
Requirement already satisfied: h11>=0.16 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub->open_clip_torch)
(0.16.0)
Requirement already satisfied: colorama in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from tqdm->open_clip_torch) (0.4.6)
Requirement already satisfied: MarkupSafe>=2.0 in
```

```
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from jinja2->torch>=2.0->open_clip_torch) (3.0.3)
Requirement already satisfied: numpy in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from torchvision->open_clip_torch) (2.4.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from torchvision->open_clip_torch) (11.3.0)
Requirement already satisfied: click>=8.2.1 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from typer->huggingface-hub->open_clip_torch) (8.3.1)
Requirement already satisfied: shellingham>=1.3.0 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from typer->huggingface-hub->open_clip_torch) (1.5.4)
Requirement already satisfied: rich>=12.3.0 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from typer->huggingface-hub->open_clip_torch) (14.3.3)
Requirement already satisfied: annotated-doc>=0.0.2 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from typer->huggingface-hub->open_clip_torch) (0.0.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from rich>=12.3.0->typer->huggingface-hub->open_clip_torch) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from rich>=12.3.0->typer->huggingface-hub->open_clip_torch) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from markdown-it-py>=2.2.0->rich>=12.3.0->typer->huggingface-
hub->open_clip_torch) (0.1.2)
Downloading open_clip_torch-3.3.0-py3-none-any.whl (1.5 MB)
   ---------------------------------------- 0.0/1.5 MB ? eta -:--:--
   ---------------------------------------- 1.5/1.5 MB 20.8 MB/s  0:00:00
Downloading timm-1.0.26-py3-none-any.whl (2.6 MB)
   ---------------------------------------- 0.0/2.6 MB ? eta -:--:--
   ---------------------------------------- 2.6/2.6 MB 36.1 MB/s  0:00:00
Downloading ftfy-6.3.1-py3-none-any.whl (44 kB)
Installing collected packages: ftfy, timm, open_clip_torch

   ---------------------------------------- 0/3 [ftfy]
   ------------ --------------------------- 1/3 [timm]
   ------------ --------------------------- 1/3 [timm]
   ------------ --------------------------- 1/3 [timm]
   ------------ --------------------------- 1/3 [timm]
   ------------ --------------------------- 1/3 [timm]
   ------------ --------------------------- 1/3 [timm]
   ------------ --------------------------- 1/3 [timm]
   ------------ --------------------------- 1/3 [timm]
```

```
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
------------- -------------------------- 1/3 [timm]
-------------------------- ------------- 2/3 [open_clip_torch]
-------------------------- ------------- 2/3 [open_clip_torch]
-------------------------- ------------- 2/3 [open_clip_torch]
---------------------------------------- 3/3 [open_clip_torch]

Successfully installed ftfy-6.3.1 open_clip_torch-3.3.0 timm-1.0.26

[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

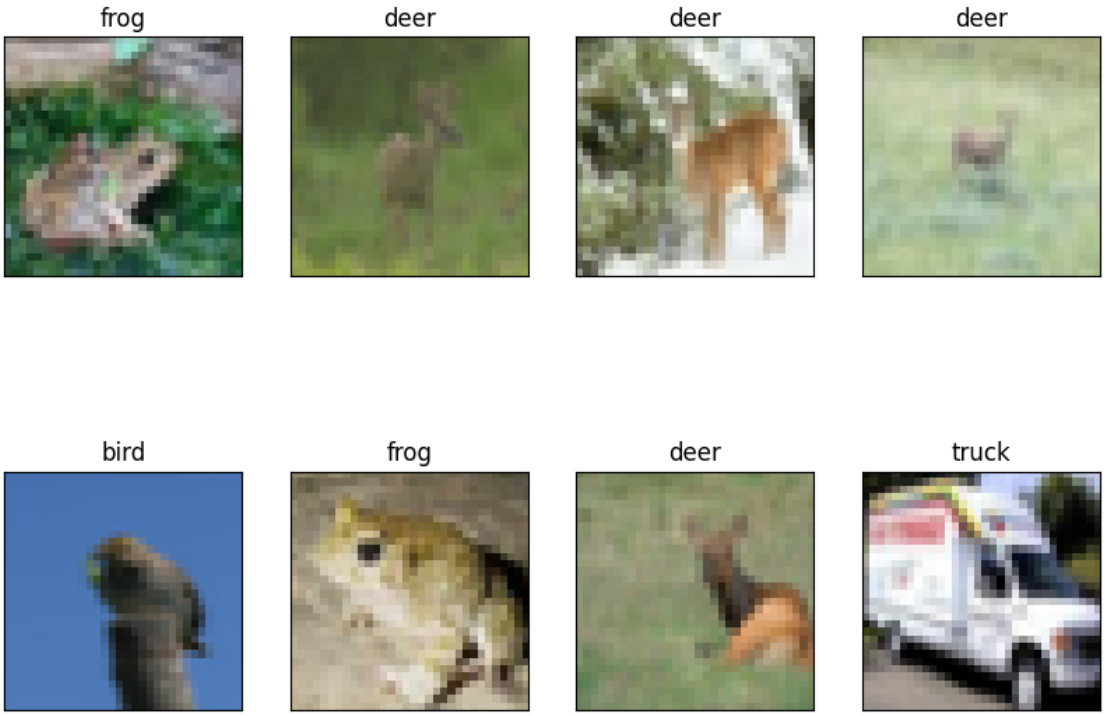
---

[3]:

```python
import torch
import torchvision
import torchvision.transforms as transforms
import transformers
from tqdm.notebook import tqdm
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
import requests
import timm
import math

batch_size = 100

device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(device)
```

```
transform = transforms.Compose([
    transforms.Resize(32),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                          shuffle=True, num_workers=4)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
                                         shuffle=False, num_workers=4)

classes = ('plane', 'car', 'bird', 'cat',
           'deer', 'dog', 'frog', 'horse', 'ship', 'truck')

def unnormalize(img, mean, std):
    return img.mul_(std.reshape(-1, 1, 1)).add_(mean.reshape(-1, 1, 1))

# utility function to plot gallery of images
def plot_gallery(images, titles, height, width, n_row=2, n_col=4):
    plt.figure(figsize=(2* n_col, 3 * n_row))
    plt.subplots_adjust(bottom=0, left=0.01, right=0.99, top=0.90, hspace=0.35)
    for i in range(n_row * n_col):
        plt.subplot(n_row, n_col, i + 1)
        curr_img = images[i]
        curr_img = unnormalize(curr_img, mean=torch.Tensor([0.485, 0.456, 0.
 ↪406]), std=torch.Tensor([0.229, 0.224, 0.225]))
        plt.imshow(np.transpose(curr_img, (1, 2, 0)))
        plt.title(classes[titles[i]], size=12)
        plt.xticks(())
        plt.yticks(())


# get some random training images
dataiter = iter(trainloader)
images, labels = next(dataiter)

# visualize some of the images of the CIFAR10 dataset
plot_gallery(images[:8], labels[:8], 32, 32, 2, 4)
```

```
cuda
```



1.0.3 *(a) Training framework for CIFAR10* (10pt)

- For the more challenging CIFAR10 dataset, it is more useful to track the training and testing set accuracies during training.
- However, unlike with MNIST and the simple linear models, it is not always possible to load the entire dataset and pass the full batch through the model to compute the accuracy.
- Hence, we need to compute the accuracy in a mini-batch manner in the function `get_accuracy_cifar10`.
  - It takes as input the `model`, a `dataloader` (either for training or test set), and the `batch_size`.
- Following this, implement the training function `train_torch_model_cifar10` (similar to MNIST) but compute and print the training and testing accuracy after every epoch.
  - It should take as input the `model`, `trainloader`, `testloader`, `batch_size`, a predefined `optimizer`, the loss function `criterion`, maximum number of epochs `max_epochs`, and `tolerance`.
- You should be able to use the `train_torch_model` code from Part I by adding support for dataloaders.

[4]:
```
def get_accuracy_cifar10(model, dataloader, batch_size):
    correct = 0
    total = 0
    with torch.no_grad():
        for X_batch, y_batch in dataloader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
```

```
            outputs = model(X_batch)
            _, predicted = torch.max(outputs.data, 1)
            total += y_batch.size(0)
            correct += (predicted == y_batch).sum().item()
    acc = 100 * correct / total
    return acc

# Define a function train_torch_model_cifar10
def train_torch_model_cifar10(model, trainloader, testloader, batch_size,␣
 ↪optimizer, criterion, max_epochs, tolerance, device='cuda'):
  losses = []
  train_acc = []
  test_acc = []
  prev_loss = float('inf')

  #######
  # 3. move model to device
  model.to(device)

  for epoch in tqdm(range(max_epochs)):
    for idx, (X_train_batch, y_train_batch) in enumerate(trainloader):
      # 3. move mini-batch data to device
      X_train_batch, y_train_batch = X_train_batch.to(device), y_train_batch.
 ↪to(device)

      # 4. reset gradients
      optimizer.zero_grad()

      # 5. prediction on mini-batch data
      pred = model(X_train_batch)

      # 6. calculate loss
      loss = criterion(pred, y_train_batch)

      # 7. backpropagate loss
      loss.backward()

      # 8. perform a single gradient update step
      optimizer.step()

    train_acc_curr = get_accuracy_cifar10(model, trainloader, batch_size)
    test_acc_curr = get_accuracy_cifar10(model, testloader, batch_size)
    #######
    # log loss every 100th epoch and print every 5000th epoch:
    losses.append((epoch, loss.item()))
    print('Epoch: {}, Loss: {}'.format(epoch, loss.item()))
```

```
    print("Train accuracy:", train_acc_curr)
    print("Test accuracy:", test_acc_curr)
    train_acc.append((epoch, train_acc_curr))
    test_acc.append((epoch, test_acc_curr))

    # break if decrease in loss is less than threshold
    if epoch > 0:
        loss_diff = prev_loss - loss.item()
        if loss_diff < tolerance:
            print(f"Early stopping at epoch {epoch}")
            break
    prev_loss = loss.item()

  # return updated model and logged losses
  return model, losses, train_acc, test_acc
```

1.0.4 *(b) ResNet* (10pt)

#1. Training with random initialization

- Using the defined `train_torch_model_cifar10`, we will first train a model using the ResNet50 architecture from scratch.
- Hyperparameters:
  - Batch size of 100
  - SGD optimizer with learning rate 0.001 and momentum 0.9
  - CE Loss
  - Train the model for a maximum of 15 epochs.
- Use the `torchvision.models.resnet50` function to define the model and set the arguments such that pretrained weights are not used.
- The default `resnet50` model has 1000 output neurons (since ImageNet1k has 1000 classes) in the last layer while we need only 10 for CIFAR10.

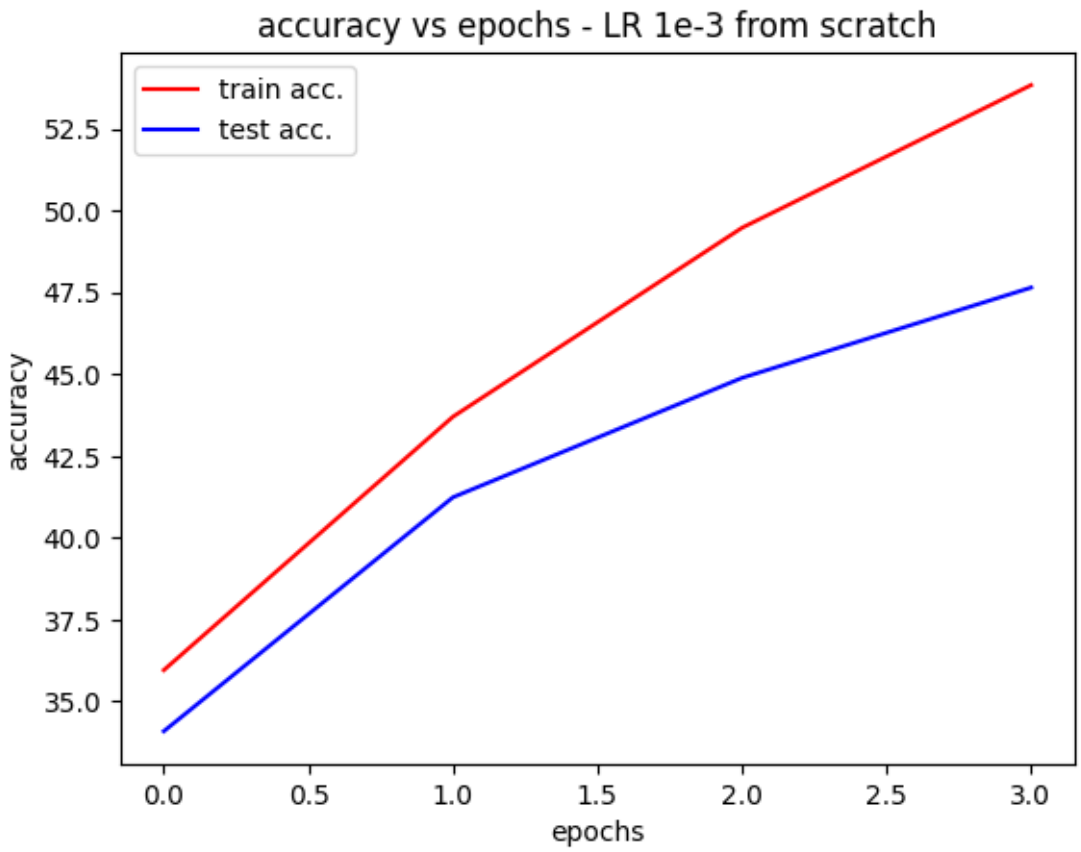
We will also compute the training and testing set accuracies after every epoch and plot them.

[7]:
```
from torchvision import models

model = models.resnet50(weights=None, num_classes=10)

batch_size = 100
# define optimizer (use torch.optim.SGD (Stochastic Gradient Descent))
# Set learning rate to lr and also set model parameters
optimizer = torch.optim.SGD(model.parameters(), lr=0.001, momentum=0.9)

model, losses, train_accs, test_accs = train_torch_model_cifar10(model,␣
 ↪trainloader, testloader, batch_size, optimizer, torch.nn.CrossEntropyLoss(),␣
 ↪max_epochs=15, tolerance=1e-6, device=device)
```

```
  0%|          | 0/15 [00:00<?, ?it/s]
```

```
Epoch: 0, Loss: 2.0099480152130127
Train accuracy: 35.958
Test accuracy: 34.09
Epoch: 1, Loss: 1.700219750404358
Train accuracy: 43.702
Test accuracy: 41.24
Epoch: 2, Loss: 1.376173973083496
Train accuracy: 49.478
Test accuracy: 44.89
Epoch: 3, Loss: 1.4372773170471191
Train accuracy: 53.84
Test accuracy: 47.65
Early stopping at epoch 3
```

[8]:
```
plt.plot([x[0] for x in train_accs],[x[1] for x in train_accs], 'r',␣
 ↪label='train acc.')
plt.plot([x[0] for x in test_accs],[x[1] for x in test_accs], 'b', label='test␣
 ↪acc.')
plt.xlabel('epochs')
plt.ylabel('accuracy')
plt.title('accuracy vs epochs - LR 1e-3 from scratch')
plt.legend()
plt.show()
```



#2. Training with pretrained initialization

- Using the defined `train_torch_model_cifar10`, we will now train a ResNet50 model initialized from an ImageNet pretrained model.
- We will use the same hyperparameters as the previous part #2.1.
  - Batch size of 100
  - SGD optimizer with learning rate 0.001 and momentum 0.9
  - CE Loss
  - Train the model for a maximum of 15 epochs.
- We will use the same `torchvision.models.resnet50` function to define the model, but set the arguments such that ImageNet1k-v2 pretrained weights are used for initialization.

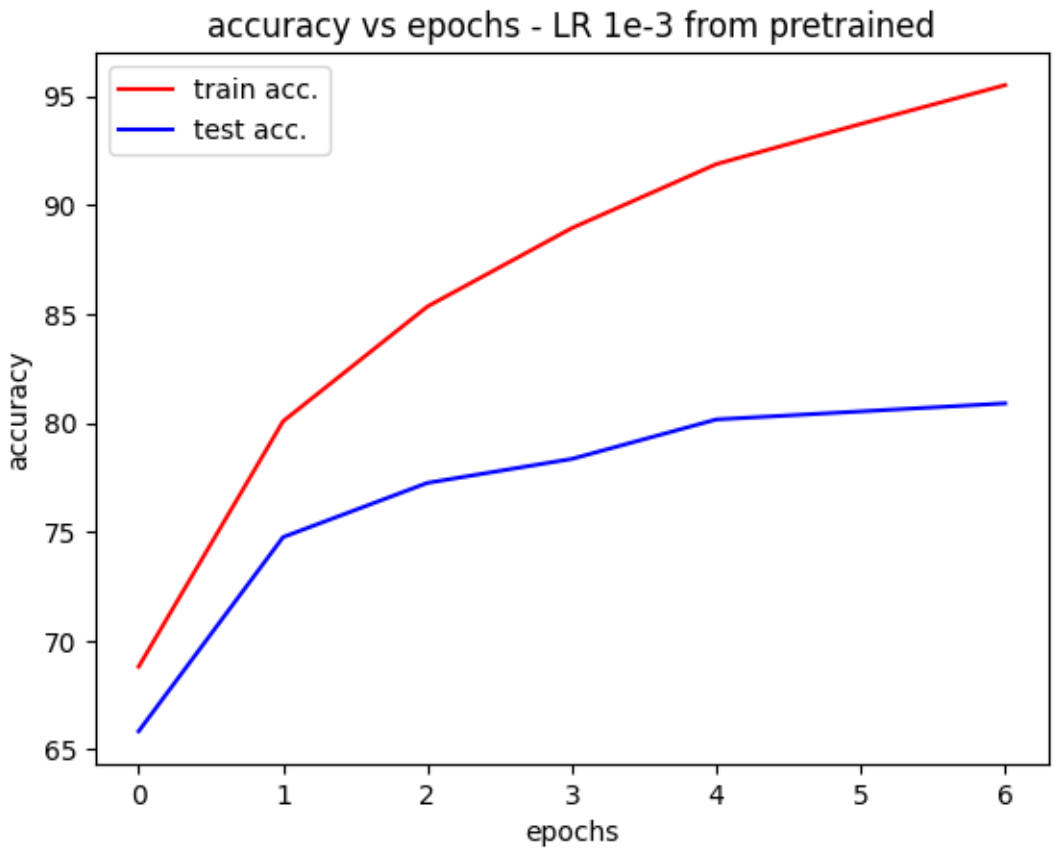
We will also compute and plot the training and testing set accuracies after every epoch.

[29]:
```
model = models.resnet50(weights='IMAGENET1K_V2')
model.fc = torch.nn.Linear(model.fc.in_features, 10)

optimizer = torch.optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
```

```
model, losses, train_accs, test_accs = train_torch_model_cifar10(model,␣
↪trainloader, testloader, batch_size, optimizer, torch.nn.CrossEntropyLoss(),␣
↪max_epochs=15, tolerance=1e-6, device=device)
```

```
  0%|          | 0/15 [00:00<?, ?it/s]
Epoch: 0, Loss: 0.940744161605835
Train accuracy: 68.804
Test accuracy: 65.84
Epoch: 1, Loss: 0.751758337020874
Train accuracy: 80.068
Test accuracy: 74.75
Epoch: 2, Loss: 0.5851027965545654
Train accuracy: 85.336
Test accuracy: 77.24
Epoch: 3, Loss: 0.5739855766296387
Train accuracy: 88.936
Test accuracy: 78.34
Epoch: 4, Loss: 0.262912780046463
Train accuracy: 91.87
Test accuracy: 80.15
Epoch: 5, Loss: 0.23424674570560455
Train accuracy: 93.714
Test accuracy: 80.52
Epoch: 6, Loss: 0.2616279125213623
Train accuracy: 95.498
Test accuracy: 80.89
Early stopping at epoch 6
```

[30]:
```
plt.plot([x[0] for x in train_accs],[x[1] for x in train_accs], 'r',␣
↪label='train acc.')
plt.plot([x[0] for x in test_accs],[x[1] for x in test_accs], 'b', label='test␣
↪acc.')
plt.xlabel('epochs')
plt.ylabel('accuracy')
plt.title('accuracy vs epochs - LR 1e-3 from pretrained')
plt.legend()
plt.show()
```



1.0.5 *(c) Vision Transformer* (https://arxiv.org/abs/2010.11929) (10pt)

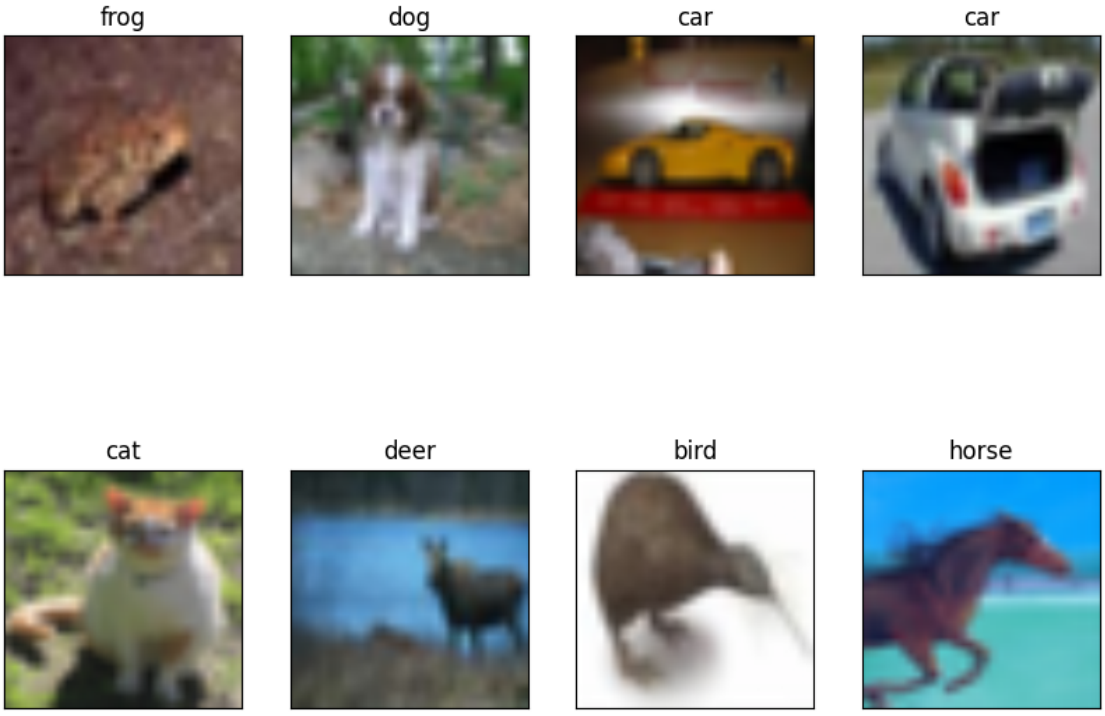
- This architecture was proposed following the popularization of transformer models for language modeling.
- Vision Transformers can be considered a variation of transformers with image inputs.
- The images are divided as 16x16 patches (the size varies with different models) and a sequence of these patches is passed as the input to the transformer model.

First, we modify the shape of the images to be 224x224 as this is what the ViT model we are using expects.

[14]:
```
vit_transform = transforms.Compose([
    transforms.Resize(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

vit_trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                            download=True, transform=vit_transform)
```

```
vit_trainloader = torch.utils.data.DataLoader(vit_trainset,␣
 ↪batch_size=batch_size,
                                               shuffle=True, num_workers=2)

vit_testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                           download=True, transform=vit_transform)
vit_testloader = torch.utils.data.DataLoader(vit_testset, batch_size=batch_size,
                                             shuffle=False, num_workers=2)

dataiter = iter(vit_trainloader)
vit_images, vit_labels = next(dataiter)

# visualize some of the images of the CIFAR10 dataset
plot_gallery(vit_images[:8], vit_labels[:8], 32, 32, 2, 4)
```

```
c:\Users\TangyKiwi\AppData\Local\Programs\Python\Python313\Lib\site-
packages\torchvision\datasets\cifar.py:83: VisibleDeprecationWarning: dtype():
align should be passed as Python or NumPy boolean but got `align=0`. Did you
mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
  entry = pickle.load(f, encoding="latin1")
```



#1. Training with random initialization

- Similarly, now we will use the `train_torch_model_cifar10` function to train a Classifier using the Vision Transformer Architecture
- Hyperparameters:
  - Batch size of 100
  - SGD optimizer with learning rate 0.001 and momentum 0.9
  - CE Loss
  - Train the model for a maximum of 7 epochs as training this model is slower than ResNet50.
- Use the `timm/efficientvit_m1.r224_in1k` model either using the `timm` library. We choose this model due to its small number of parameters (3M) which make it easy to train and experiment with.
- By setting the `num_classes` argument, we can customise the number of required classes which is 10, in the case of CIFAR10.

We will also compute the training and testing set accuracies after every epoch and plot them.

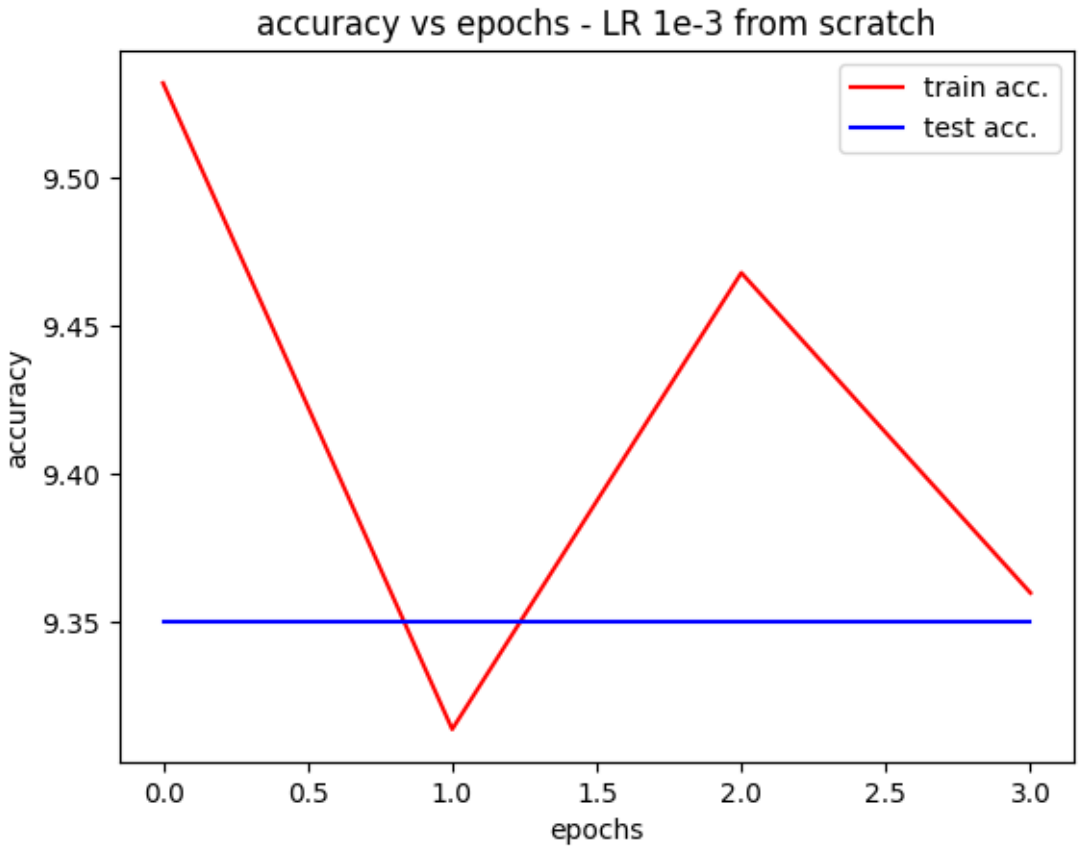
Note: This function may take approx 15 minutes to run. Ensure it works on a non-GPU runtime first to ensure you don't exhaust resources.

[19]:
```
import torch.nn as nn
import timm

model = timm.create_model('efficientvit_m1.r224_in1k', pretrained=False,␣
 ↪num_classes=10)

model, losses, train_accs, test_accs = train_torch_model_cifar10(model,␣
 ↪vit_trainloader, vit_testloader, batch_size, optimizer, torch.nn.
 ↪CrossEntropyLoss(), max_epochs=7, tolerance=1e-6, device=device)
```

```
  0%|          | 0/7 [00:00<?, ?it/s]
Epoch: 0, Loss: 2.371126651763916
Train accuracy: 9.532
Test accuracy: 9.35
Epoch: 1, Loss: 2.341001510620117
Train accuracy: 9.314
Test accuracy: 9.35
Epoch: 2, Loss: 2.322421073913574
Train accuracy: 9.468
Test accuracy: 9.35
Epoch: 3, Loss: 2.3620872497558594
Train accuracy: 9.36
Test accuracy: 9.35
Early stopping at epoch 3
```

[20]:
```
plt.plot([x[0] for x in train_accs],[x[1] for x in train_accs], 'r',␣
 ↪label='train acc.')
plt.plot([x[0] for x in test_accs],[x[1] for x in test_accs], 'b', label='test␣
 ↪acc.')
```

```
plt.xlabel('epochs')
plt.ylabel('accuracy')
plt.title('accuracy vs epochs - LR 1e-3 from scratch')
plt.legend()
plt.show()
```



#2. Training with pretrained initialization

- We will use the same `timm/efficientvit_m1.r224_in1k` model but we will use its pretrained weights on ImageNet using the `pretained` parameter.
- We will use the same hyperparameters as the previous part #3.1.
  - Batch size of 100
  - SGD optimizer with learning rate 0.001 and momentum 0.9
  - CE Loss
  - Train the model for a maximum of 7 epochs.

We will also compute and plot the training and testing set accuracies after every epoch.

Note: This function may take approx 15 minutes to run. Ensure it works on a non-GPU runtime first to ensure you don't exhaust resources.

[15]:
```
import torch.nn as nn
import timm

model = timm.create_model('efficientvit_m1.r224_in1k', pretrained=True,␣
 ↪num_classes=10)
# define optimizer (use torch.optim.SGD (Stochastic Gradient Descent))
# Set learning rate to lr and also set model parameters
optimizer = torch.optim.SGD(model.parameters(), lr=0.001, momentum=0.9)

model, losses, train_accs, test_accs = train_torch_model_cifar10(model,␣
 ↪vit_trainloader, vit_testloader, batch_size, optimizer, torch.nn.
 ↪CrossEntropyLoss(), max_epochs=7, tolerance=1e-6, device=device)
```

```
model.safetensors:   0%|          | 0.00/12.1M [00:00<?, ?B/s]
```

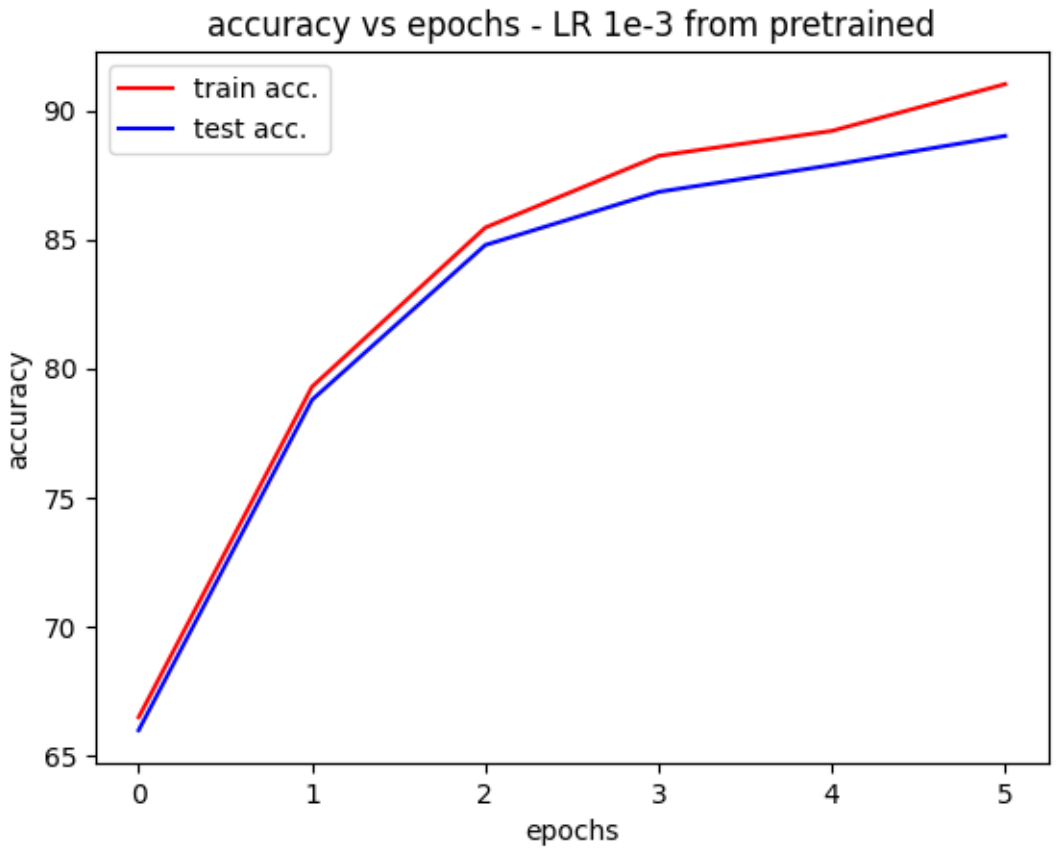
```
c:\Users\TangyKiwi\AppData\Local\Programs\Python\Python313\Lib\site-
packages\huggingface_hub\file_download.py:129: UserWarning: `huggingface_hub`
cache-system uses symlinks by default to efficiently store duplicated files but
your machine does not support them in
C:\Users\TangyKiwi\.cache\huggingface\hub\models--timm--
efficientvit_m1.r224_in1k. Caching files will still work but in a degraded
version that might require more space on your disk. This warning can be disabled
by setting the `HF_HUB_DISABLE_SYMLINKS_WARNING` environment variable. For more
details, see https://huggingface.co/docs/huggingface_hub/how-to-
cache#limitations.
To support symlinks on Windows, you either need to activate Developer Mode or to
run Python as an administrator. In order to activate developer mode, see this
article: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-
device-for-development
  warnings.warn(message)
```

```
  0%|          | 0/7 [00:00<?, ?it/s]
```

```
Epoch: 0, Loss: 1.1742831468582153
Train accuracy: 66.484
Test accuracy: 65.98
Epoch: 1, Loss: 0.720608651638031
Train accuracy: 79.286
Test accuracy: 78.78
Epoch: 2, Loss: 0.45442187786102295
Train accuracy: 85.456
Test accuracy: 84.78
Epoch: 3, Loss: 0.3313128650188446
Train accuracy: 88.238
Test accuracy: 86.84
Epoch: 4, Loss: 0.24218089878559113
Train accuracy: 89.204
Test accuracy: 87.88
Epoch: 5, Loss: 0.25919994711875916
```

```
Train accuracy: 91.018
Test accuracy: 89.01
Early stopping at epoch 5
```

[18]:
```
plt.plot([x[0] for x in train_accs],[x[1] for x in train_accs], 'r',␣
 ↪label='train acc.')
plt.plot([x[0] for x in test_accs],[x[1] for x in test_accs], 'b', label='test␣
 ↪acc.')
plt.xlabel('epochs')
plt.ylabel('accuracy')
plt.title('accuracy vs epochs - LR 1e-3 from pretrained')
plt.legend()
plt.show()
```



1.0.6 *(d) CLIP (Contrastive Language-Image Pretraining - Paper)* (10pt)

- CLIP jointly trains a text-encoder and image-encoder on images and text captions.
- CLIP generates representations (vector embeddings) of both the text and image such that they lie in the same vector space.
- CLIP optimizes the embeddings such that the image and captions are closer in the vector

space if they correspond to each other and farther if they do not.
- The image-encoder is a pluggable model - usually a ResNet or Vision Transformer. We will use both.
- For this section, we will not be doing any pretraining or finetuning, rather just loading and evaluating it on CIFAR10 test-set (zero-shot image classification).

Note: This is a simplified explanation. You can read the paper if you need more details. However, this is not essential for the assignment.

[21]:
```
import open_clip

captions = [f"a photo of a {label}" for label in classes]

clip_testset = torchvision.datasets.CIFAR10(root='./data', download=True,␣
 ↪train=False)
```

```
c:\Users\TangyKiwi\AppData\Local\Programs\Python\Python313\Lib\site-
packages\torchvision\datasets\cifar.py:83: VisibleDeprecationWarning: dtype():
align should be passed as Python or NumPy boolean but got `align=0`. Did you
mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
  entry = pickle.load(f, encoding="latin1")
```

Note: We will use the `open_clip` package to run CLIP. You can refer the documentation at https://github.com/mlfoundations/open_clip

We define a new testset for CLIP since it uses PIL images as inputs and does not require tensor conversion

Note: We convert the class labels to a caption format which works better with CLIP.

#1. Predict and accuracy function

- Write the `clip_predict` function that takes in the CLIP `model`, a list of processed `images`, tokenized `texts` and encodes the texts and images.
- The function should then use the encoded features to find and return the predicted label for each of the `images`.
- Complete the `clip_accuracy` function takes in the CLIP `model`, the CLIP image `processor` and text/caption `tokenizer` and torchvision image `dataset` as input
- The function processes the images and texts into vectors in batch-form and uses the `clip_predict` predictions to calculate accuracy over the `dataset`

[22]:
```
def clip_predict(model, images, texts):

  with torch.no_grad():
      # get the image and text vectors using the model's encode functions
      image_vectors = model.encode_image(images)
      text_vectors = model.encode_text(texts)

      # normalize the image and text vectors
```

```
        image_vectors = image_vectors / image_vectors.norm(dim=-1, keepdim=True)
        text_vectors = text_vectors / text_vectors.norm(dim=-1, keepdim=True)

        # find the cosine similarity between the image and text vectors and␣
 ↪softmax them to get the probabilities
        logits = model.logit_scale.exp() * image_vectors @ text_vectors.t()
        probs = logits.softmax(dim=-1)

        # calculate predicted label index
        preds = probs.argmax(dim=-1)

    #return predicted labels
    return preds

def clip_accuracy(model, processor, tokenizer, dataset, batch_size,␣
 ↪captions=captions):

    # process the text using the tokenizer
    texts = tokenizer(captions).to(device)

    #calculate the number of batches
    number_of_batches = math.ceil(len(dataset) / batch_size)

    correct = 0

    for i in range(number_of_batches):

      # define start and end indices
      start = i * batch_size
      end = min((i + 1) * batch_size, len(dataset))

      # process the images in the batch using the processor
      images = torch.stack([processor(dataset[j][0]) for j in range(start, end)]).
 ↪to(device)

      # get the labels
      labels = torch.tensor([dataset[j][1] for j in range(start, end)]).to(device)

      # use the clip_predict function to
      preds = clip_predict(model, images, texts)
      correct = correct + (preds == labels).sum().item()

    return correct / len(dataset)
```

#2. CLIP Model using Vision Transformer

- Use the `open_clip.create_model_and_transforms` function from the `open_clip` library to

load the model and image processor.

- Pass the `ViT-B-32` as the model and `openai` as the pretrained weights to use.
- Use the `open_clip.get_tokenizer` with the same model name to get the tokenizer.

[25]:
```
model,_, processor = open_clip.create_model_and_transforms('ViT-B-32',␣
↪pretrained='openai')
model.to(device)
tokenizer = open_clip.get_tokenizer('ViT-B-32')
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated
requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and
faster downloads.
c:\Users\TangyKiwi\AppData\Local\Programs\Python\Python313\Lib\site-
packages\open_clip\factory.py:450: UserWarning: QuickGELU mismatch between final
model config (quick_gelu=False) and pretrained tag 'openai' (quick_gelu=True).
  warnings.warn(
```

- Call the `clip_accuracy` function with appropriate parameters. Optionally, you can pass in custom images and captions to the `clip_predict` function to see interesting outputs.

[26]:
```
clip_accuracy(model, processor, tokenizer, clip_testset, batch_size, captions)
```

[26]:
```
0.8648
```

#3. CLIP Model using Resnet50

- We are now going to use a ResNet50 based CLIP-model instead of Vision Transformer a
- Use the same code as in #2 except the model name should be `RN50`

[27]:
```
model,_, processor = open_clip.create_model_and_transforms('RN50',␣
↪pretrained='openai')
model.to(device)
tokenizer = open_clip.get_tokenizer('RN50')
```

```
open_clip_model.safetensors:   0%|          | 0.00/408M [00:00<?, ?B/s]

c:\Users\TangyKiwi\AppData\Local\Programs\Python\Python313\Lib\site-
packages\huggingface_hub\file_download.py:129: UserWarning: `huggingface_hub`
cache-system uses symlinks by default to efficiently store duplicated files but
your machine does not support them in
C:\Users\TangyKiwi\.cache\huggingface\hub\models--timm--resnet50_clip.openai.
Caching files will still work but in a degraded version that might require more
space on your disk. This warning can be disabled by setting the
`HF_HUB_DISABLE_SYMLINKS_WARNING` environment variable. For more details, see
https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations.
To support symlinks on Windows, you either need to activate Developer Mode or to
run Python as an administrator. In order to activate developer mode, see this
article: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-
```

```
device-for-development
  warnings.warn(message)
c:\Users\TangyKiwi\AppData\Local\Programs\Python\Python313\Lib\site-
packages\open_clip\factory.py:450: UserWarning: QuickGELU mismatch between final
model config (quick_gelu=False) and pretrained tag 'openai' (quick_gelu=True).
  warnings.warn(
```

- Call the `clip_accuracy` function with appropriate parameters here. Optionally, you can pass in custom images and captions to the `clip_predict` function to see interesting outputs.

[28]:
```
clip_accuracy(model, processor, tokenizer, clip_testset, batch_size, captions)
```

[28]:
```
0.353
```

1.0.7 *(e) Analysis* (5pt)

Effect of pretraining:
ResNet50 random initialization resulted in a final train accuracy of 53.84 and test accuracy of 47.65. Using the IMAGENET1K_V2 pretrained weights resulted in a final train accuracy of 95.498 and test accuracy of 80.89. This is a significant improvement. The vision transformer model with no pretraining resulted in a final train accuracy of 9.36 and test accuracy of 9.35. This is actually worse than random guessing the 10 classes. But with pretrainig, the final train accuracy was 91.018 and final test accuracy was 89.01. This shows that pretraining is critical for performance in ViT's, which makes sense as they don't have built in inductive biases like locality in the ResNet.

Effect of architecture:
The ViT model performed the best. The ResNet50 model learns local spacial features and works well for the small datset of CIFAR-10, and provides a good baseline benchmark model. ViT on the other hand, learns global relationships via attention and is able to capture complex patterns, but requires pretraining to work properly. This could also be the reason behind why it performs the best. The CLIP models enable zero-shot classification but its performance depends heavily on the backbone, hence why the ViT CLIP model performs significantly better than the ResNet50 CLIP model. Transformer-based models and multimodal models achieve superior performance when pretrained on large datasets, while CNNs remain more robust in low-data settings.

PA-Q1-Part III

April 15, 2026

1 *PA - Part III: Training a Robust Model - Optional/Bonus* (10pt)

Keywords: Adversarial Robustness Training

About the dataset:
The MNIST database (Modified National Institute of Standards and Technology database) is a large database of handwritten digits that is commonly used for training various image processing systems.
The MNIST database contains 70,000 labeled images. Each datapoint is a 28×28 pixels grayscale image.

Agenda: * The PA is split into three parts, the first part dealing with miniature models which we will build from scratch, the second part dealing with modern architectures and the bonus third part dealing with training vision models robust to adversarial attacks. * In this part, you will train a 2-hidden layer neural network which is robust to adversarial attacks. * You will train models on adversarial examples generated using FGSM and PGD.

Note: * Hardware acceleration (GPU) is recommended but not required for this part. * A note on working with GPU: * Take care that whenever declaring new tensors, set `device=device` in parameters. * You can also move a declared torch tensor/model to device using `.to(device)`. * To move a torch model/tensor to cpu, use `.to('cpu')` * Keep in mind that all the tensors/model involved in a computation have to be on the same device (CPU/GPU). * Run all the cells in order. * Only **add your code** to cells marked with "TODO:" or with "…" * You should not have to change variable names where provided, but you are free to if required for your implementation.

1.0.1 *Setup: Imports and Utils*

[1]:
```
# install this library
!pip install gdown
```

```
Collecting gdown
  Downloading gdown-6.0.0-py3-none-any.whl.metadata (7.4 kB)
Requirement already satisfied: beautifulsoup4 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from gdown) (4.14.2)
Requirement already satisfied: filelock in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from gdown) (3.25.0)
Requirement already satisfied: requests[socks] in
```

```
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from gdown) (2.32.5)
Requirement already satisfied: tqdm in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from gdown) (4.67.3)
Requirement already satisfied: soupsieve>1.2 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from beautifulsoup4->gdown) (2.8)
Requirement already satisfied: typing-extensions>=4.0.0 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from beautifulsoup4->gdown) (4.15.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from requests[socks]->gdown) (3.4.3)
Requirement already satisfied: idna<4,>=2.5 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from requests[socks]->gdown) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from requests[socks]->gdown) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from requests[socks]->gdown) (2026.2.25)
Collecting PySocks!=1.5.7,>=1.5.6 (from requests[socks]->gdown)
  Downloading PySocks-1.7.1-py3-none-any.whl.metadata (13 kB)
Requirement already satisfied: colorama in
c:\users\tangykiwi\appdata\local\programs\python\python313\lib\site-packages
(from tqdm->gdown) (0.4.6)
Downloading gdown-6.0.0-py3-none-any.whl (18 kB)
Downloading PySocks-1.7.1-py3-none-any.whl (16 kB)
Installing collected packages: PySocks, gdown

   ---------------------- ------------------- 1/2 [gdown]
   ---------------------------------------- 2/2 [gdown]

Successfully installed PySocks-1.7.1 gdown-6.0.0


[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

[3]:
```python
# imports
import torch
import torch.nn as nn
import numpy as np
import requests
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

```
import matplotlib.pyplot as plt
from tqdm.notebook import tqdm
import gdown
from zipfile import ZipFile

# set device
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print(device)

# loading the dataset full MNIST dataset
mnist_train = datasets.MNIST("./data", train=True, download=True,␣
 ↪transform=transforms.ToTensor())
mnist_test = datasets.MNIST("./data", train=False, download=True,␣
 ↪transform=transforms.ToTensor())

mnist_train.data = mnist_train.data.to(device)
mnist_test.data = mnist_test.data.to(device)

mnist_train.targets = mnist_train.targets.to(device)
mnist_test.targets = mnist_test.targets.to(device)

# reshape and min-max scale
X_train =  (mnist_train.data.reshape((mnist_train.data.shape[0], -1))/255).
 ↪to(device)
y_train = mnist_train.targets
X_test = (mnist_test.data.reshape((mnist_test.data.shape[0], -1))/255).
 ↪to(device)
y_test = mnist_test.targets

# first few examples
example_data = mnist_test.data[:18]/255
example_data_flattened  = example_data.view((example_data.shape[0], -1)).
 ↪to(device) # needed for training
example_labels = mnist_test.targets[:18].to(device)
```

```
cuda:0
```

1.0.2 Adversarial training:

- To train robust models, the most intuitive strategy is to train on adversarial examples.
- The adversarial (robust) loss function is defined as: $\min_{\theta} \frac{1}{|S|} \sum_{x,y \in S} \max_{\|\delta\| \leq \epsilon} \ell(h_\theta(x+\delta), y)$, where θ are the learnable parameters, S is the set of training examples with x representing the input example and y the ground truth label, h_θ is the score function (neural network model), δ is the attack perturbation, and ϵ is the attack budget.

- This is also known as the min-max loss function. The gradient descent step now becomes: $\theta := \theta - \frac{\alpha}{|B|} \sum_{x,y \in B} \nabla_\theta \max_{\|\delta\| \leq \epsilon} \ell(h_\theta(x+\delta), y)$,
  where B is the mini-batch and α is the learning rate.
- Now the question becomes how to solve the inner term: $\nabla_\theta \max_{\|\delta\| \leq \epsilon} \ell(h_\theta(x+\delta), y)$ of the gradient descent step.
- For this, we can use **Danskin's Theorem**, which states that to compute the (sub)gradient of a function containing a max term, we need to simply
  1. find the maximum and,
  2. compute the normal gradient evaluated at this point.
- This holds only when you have the exact maximum. Note that it is not possible to solve the inner maximization problem exactly (NP-hard). However, the better job we do of solving the inner maximization problem, the closer it seems that Danskin's theorem starts to hold. That is why we can re-use methods such as FGSM/PGD to find approximate worst case examples.
- In other words, we can perform the attack to find $\delta^* = \arg\max_{\|\delta\| \leq \epsilon} \ell(h_\theta(x+\delta), y)$, and then compute this term at the perturbed image: $\nabla_\theta \ell(h_\theta(x+\delta^*), y)$.

In summary, we will create an adversarial example for each datapoint in the mini-batch and use the loss corresponding to these adversarial examples to compute the gradient.

We explore two attacks to get the perturbration - Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD)

- In the Fast Gradient Sign Method (FGSM), the perturbation δ on an input example (e.g. input image) X is given by $\epsilon \cdot \text{sign}(g)$.
  - Here, g is the gradient of the loss function $g := \nabla_\delta \ell(h_\theta(x+\delta), y)$.
  - ℓ is the loss function, more precisely `nn.CrossEntropyLoss`. In the first timestep, this value of δ is 0.
- For the Projected Gradient Descent (PGD) attack, you will create an adversarial example by iteratively performing **steepest descent** with a fixed step size α.
  - The update rule is: $\delta := P(\delta + \alpha\, \text{sign}(\nabla_\delta \ell(h_\theta(x+\delta), y)))$.
  - Here δ is the perturbation, θ are the frozen DNN parameters, x and y is the training example and its ground truth label respectively.
  - h_θ is the score function and ℓ denotes the loss function.
  - P denotes the projection onto a norm ball (l_∞, l_1, l_2, etc.) of interest. For l_∞ ball, this just means clamping the value of δ between $-\epsilon$ and ϵ.

1.0.3 *(a) Setup* (4pt)

- In this part you will create a few adversarial examples using FGSM and PGD attacks. Use an attack budget $\epsilon = 0.05$.

#1. Define the `fgsm` function

- Define a function `fgsm` which takes as input the neural network model (`model`), test examples (`X`), target labels (`y`), and the attack budget (`epsilon`).
- Return the value of the perturbation (δ) after one gradient descent step.

[4]:
```
#######
#TODO:

import torch
import torch.nn as nn

def fgsm(model, x, y, epsilon):
    delta = torch.zeros_like(x, requires_grad=True)

    output = model(x + delta)
    loss = nn.CrossEntropyLoss()(output, y)
    loss.backward()
    return epsilon * delta.grad.detach().sign()
#######
```

#2. Define the `pgd` function

- Instead of using FGSM, now use Projected Gradient Descent (PGD) with projection on l_∞ ball for the attack.
- Define a function `pgd` that takes as input the neural network model (`model`), training examples (`X`), target labels (`y`), step size (`alpha`), attack budget (`epsilon`), and number of iterations (`num_iter`).
- Return the perturbation (δ) after `num_iter` gradient descent steps.

[5]:
```
#TODO:

def pgd(model, x, y, alpha, epsilon, num_iter):
    # random sampling uniformly between 0 and 1
    delta = torch.rand_like(x)
    # bring delta in -eps to eps range
    # how? --> [0, 1] --> [0, 2*eps] --> [-eps, eps]
    delta = delta * 2 * epsilon - epsilon
    delta = delta.detach()

    for t in (range(num_iter)):
        delta.requires_grad = True

        model.zero_grad()
        output = model(x + delta)
        loss = nn.CrossEntropyLoss()(output, y)
        loss.backward()

        grad = delta.grad.detach()
        delta = delta + alpha * grad.sign()
```

```
        delta = torch.clamp(delta, -epsilon, epsilon)
        delta = torch.clamp(x + delta, 0, 1) - x
        delta = delta.detach()

    return delta
```

#3. Define a 2 hidden-layer NN in PyTorch

- Create a 2-hidden-layer neural network model in PyTorch.
- The input should be the size of the flattened MNIST image, and output layer should be of size 10, which is the number of target labels.
- Each of the two hidden layers should be of size 1024 with ReLU activations between each subsequent layer except the last layer.

You can refer to the structure of `NN_Model` from Part 1 (d) #5

[6]:
```
#######
from torch.nn.functional import relu
#TODO:
class NN_Model(nn.Module):
    def __init__(self):
        super(NN_Model, self).__init__()

        self.fc1 = nn.Linear(28*28, 1024)
        self.fc2 = nn.Linear(1024, 1024)
        self.fc3 = nn.Linear(1024, 10)

    def forward(self, X):
        X = X.view(X.shape[0], -1)
        X = relu(self.fc1(X))
        X = relu(self.fc2(X))
        X = self.fc3(X)
        return X

#######
```

#4. Adversarial Training Framework

- Define a function `train_torch_model_adversarial`
  - which takes as input: a PyTorch model (`model`), batch size (`batch_size`), loss function (`criterion`), maximum number of epochs (`max_epochs`), training data (`X_train, y_train`), learning rate (`lr`), tolerance for stopping (`tolerance`), adversarial strategy (`adversarial_strategy`: `None/'fgsm'/'pgd'`), and attack budget(`epsilon`).
  - Note: use an SGD optimizer with the given learning rate (`lr`) to update all the model parameters.
- If `adversarial_strategy` is `None`, don't train on adversarial examples.
- This function will return a tuple of `(model, losses)`, where `model` is the trained model, and `losses` are a list of tuple of loss logged every epoch.

- The only difference from the function `train_torch_model` that you wrote in Part 1 (c) #2 is that you find the adversarial noise using an attack (based on `adversarial strategy`) and add it to the input before training with it.

[10]:
```
#######
#TODO:
def train_torch_model_adversarial(model, batch_size, criterion, max_epochs, \
                                  X_train, y_train, lr, tolerance, \
                                  adversarial_strategy, epsilon):
    losses = []
    num_batches = X_train.shape[0] // batch_size
    prev_loss = float('inf')
    optimizer = torch.optim.SGD(model.parameters(), lr=lr)
    for epoch in tqdm(range(max_epochs)):
        model.train()
        epoch_loss = 0.0

        for i in range(num_batches):
            start_idx = i * batch_size
            end_idx = start_idx + batch_size
            x_batch = X_train[start_idx:end_idx].to(device)
            y_batch = y_train[start_idx:end_idx].to(device)

            if adversarial_strategy == 'fgsm':
                delta = fgsm(model, x_batch, y_batch, epsilon)
            elif adversarial_strategy == 'pgd':
                # hard coded for next part
                delta = pgd(model, x_batch, y_batch, alpha=0.01,␣
↪epsilon=epsilon, num_iter=40)
            else:
                delta = torch.zeros_like(x_batch)

            optimizer.zero_grad()
            outputs = model(x_batch + delta)
            loss = criterion(outputs, y_batch)
            loss.backward()
            optimizer.step()

            epoch_loss += loss.item()

        epoch_loss /= num_batches
        losses.append((epoch, epoch_loss))

        if abs(prev_loss - epoch_loss) < tolerance:
            print(f"Early stopping at epoch {epoch+1} with loss: {epoch_loss:.
↪4f}")
            break
```

```
        prev_loss = epoch_loss
    return model, losses

#######
```

1.0.4 *(b) Training and evaluation with different strategies (3 points)*

#1. Standard, FGSM-based, and PGD-based training

- Train three models:
  - without adversarial training,
  - with adversarial training using `fgsm`
  - with adversarial training using `pgd`.
- Hyperparameters
  - Use attack budget `epsilon` of 0.05.
  - Use a batch-size 512
  - Train for 20 epochs with learning rate 10^{-2}, and early stopping tolerance of 10^{-6}.
  - For `pgd`, use a step-size `alpha=0.01` and number of iterations `num_iter=40` when training.

  Note: PGD implementation will be slow (1hr+) when using a non-GPU runtime.

[11]:
```
#######
#TODO:
trained_models = {}
batch_size=512
tolerance = 1e-6
lr = 0.01
max_epochs = 20
epsilon = 0.05

train_with_clean = False

model = NN_Model().to(device)
criterion = nn.CrossEntropyLoss()
model, losses = train_torch_model_adversarial(
    model, batch_size, criterion, max_epochs, \
    X_train, y_train, lr, tolerance, \
    adversarial_strategy='None', epsilon=epsilon
)
trained_models['None']=(model, losses)
print('Final training loss for None model:', losses[-1][1])

model = NN_Model().to(device)
criterion = nn.CrossEntropyLoss()
model, losses = train_torch_model_adversarial(
    model, batch_size, criterion, max_epochs, \
```

```
    X_train, y_train, lr, tolerance, \
    adversarial_strategy='fgsm', epsilon=epsilon
)
trained_models['fgsm']=(model, losses)
print('Final training loss for fgsm model:', losses[-1][1])

model = NN_Model().to(device)
criterion = nn.CrossEntropyLoss()
model, losses = train_torch_model_adversarial(
    model, batch_size, criterion, max_epochs, \
    X_train, y_train, lr, tolerance, \
    adversarial_strategy='pgd', epsilon=epsilon
)
trained_models['pgd']=(model, losses)
print('Final training loss for pgd model:', losses[-1][1])

#######
```

```
  0%|          | 0/20 [00:00<?, ?it/s]
Final training loss for None model: 0.3547703732027967
  0%|          | 0/20 [00:00<?, ?it/s]
Final training loss for fgsm model: 0.8776726498563066
  0%|          | 0/20 [00:00<?, ?it/s]
Final training loss for pgd model: 0.7060453815337939
```

#2. Measuring Standard Performance

- Compute and print the accuracy of each of the three trained models on the clean test dataset.
- You can implement a function similar to the `print_accuracies_torch` function from HW1-Q1 (but compute accuracy only for the test set).

[12]:
```
#######
#TODO:

from sklearn.metrics import accuracy_score
import numpy as np

def get_accuracies_torch(model, X, y):
    model.eval()

    with torch.no_grad():
        preds = model(X)
        y_pred = torch.argmax(preds, dim=1).cpu().numpy()
    return accuracy_score(y.cpu().numpy(), y_pred)
```

```
for key in trained_models:
    model, _ = trained_models[key]
    acc = get_accuracies_torch(model, X_test, y_test)
    print('Accuracy of {} model on clean test dataset: {}'.format(key, acc))

#######
```

```
Accuracy of None model on clean test dataset: 0.9052
Accuracy of fgsm model on clean test dataset: 0.889
Accuracy of pgd model on clean test dataset: 0.8962
```

#3. Measuring Adversarial Robustness

- Using the same test dataset, perform adversarial attack to compute robust accuracy for each of the three models. Report the robust accuracy of each of the three models for both:
  - FGSM attack
  - PGD attack
- Note: In total, you need to report 6 robust accuracies here (3 models * 2 attacks).
- To create PGD attack examples, use `alpha=0.01`, `num_iter=40`.

[13]:
```
#######
#TODO:
for key in trained_models:
    model, _ = trained_models[key]

    # for fgsm
    delta = fgsm(model, X_test, y_test, epsilon)
    score = get_accuracies_torch(model, X_test + delta, y_test)
    print('Robust accuracy of {} model on fgsm attack: {}'.format(key, score))

    # for pgd
    delta = pgd(model, X_test, y_test, alpha=0.01, epsilon=epsilon, num_iter=40)
    score = get_accuracies_torch(model, X_test + delta, y_test)
    print('Robust accuracy of {} model on pgd attack: {}'.format(key, score))

#######
```

```
Robust accuracy of None model on fgsm attack: 0.5742
Robust accuracy of None model on pgd attack: 0.723
Robust accuracy of fgsm model on fgsm attack: 0.7237
Robust accuracy of fgsm model on pgd attack: 0.7845
Robust accuracy of pgd model on fgsm attack: 0.7126
Robust accuracy of pgd model on pgd attack: 0.7857
```

1.0.5 *(c) Evaluate the robust trained models at different epsilon (1.5 points)*

- Report the robust accuracy of each of the three models (standard-trained, FGSM-trained and PGD-trained from **(b)**) using FGSM and PGD attacks at `epsilon = [0, 0.01, 0.02,`

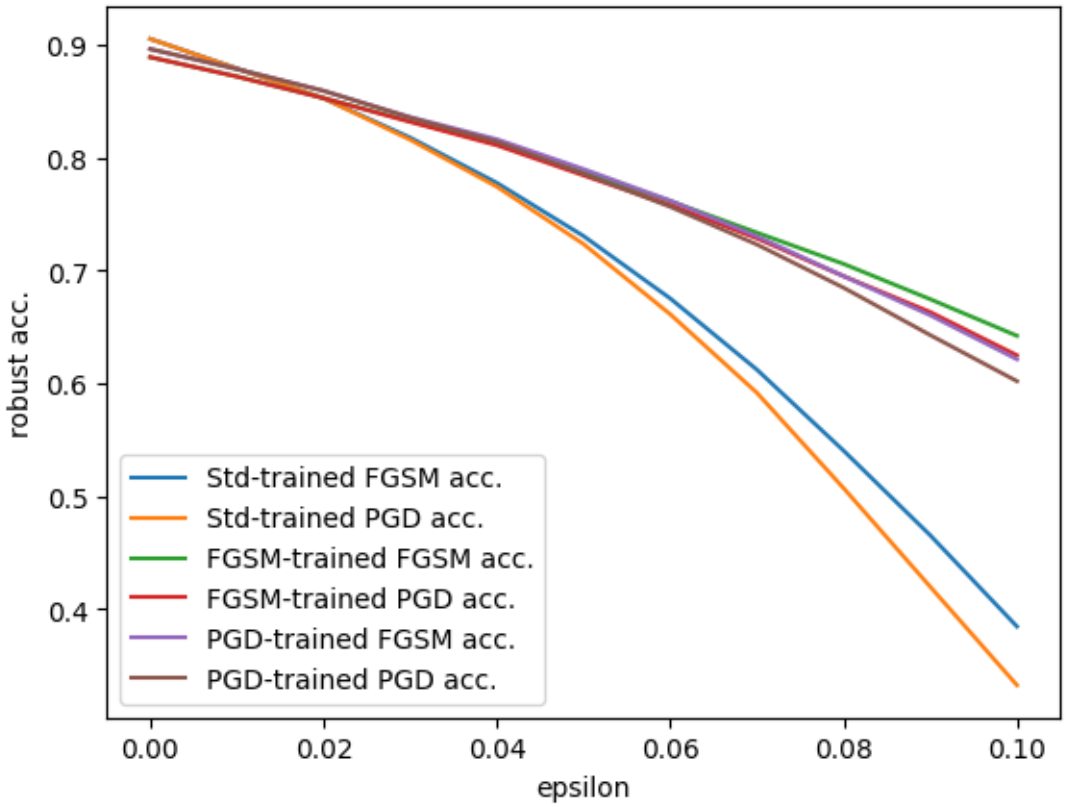
..., 0.09, 0.1].

- Plot a (single) graph of robust accuracy vs. `epsilon` (i.e. six curves, since we have 3 models and 2 attacks).

[14]:
```
#######
# TODO:

eps_list = np.arange(0, 0.11, 0.01)

std_model_fgsm = list()
std_model_pgd = list()
fgsm_model_fgsm = list()
fgsm_model_pgd = list()
pgd_model_fgsm = list()
pgd_model_pgd = list()

for eps in eps_list:
    for key in trained_models:
        model, _ = trained_models[key]
        model.eval()

        delta = fgsm(model, X_test, y_test, eps)
        X_adv = torch.clamp(X_test + delta, 0, 1)
        fgsm_score = get_accuracies_torch(model, X_adv, y_test)

        delta = pgd(model, X_test, y_test, alpha=0.01, epsilon=eps, num_iter=40)
        X_adv = torch.clamp(X_test + delta, 0, 1)
        pgd_score = get_accuracies_torch(model, X_adv, y_test)

        if key == "None":
            std_model_fgsm.append(fgsm_score)
            std_model_pgd.append(pgd_score)
        elif key == "fgsm":
            fgsm_model_fgsm.append(fgsm_score)
            fgsm_model_pgd.append(pgd_score)
        elif key == "pgd":
            pgd_model_fgsm.append(fgsm_score)
            pgd_model_pgd.append(pgd_score)

#######
```

[15]:
```
plt.plot(eps_list, std_model_fgsm, label='Std-trained FGSM acc.')
plt.plot(eps_list, std_model_pgd, label='Std-trained PGD acc.')

plt.plot(eps_list, fgsm_model_fgsm, label='FGSM-trained FGSM acc.')
plt.plot(eps_list, fgsm_model_pgd, label='FGSM-trained PGD acc.')
```

```
plt.plot(eps_list, pgd_model_fgsm, label='PGD-trained FGSM acc.')
plt.plot(eps_list, pgd_model_pgd, label='PGD-trained PGD acc.')

plt.xlabel('epsilon')
plt.ylabel('robust acc.')
plt.legend()
plt.show()
```



1.0.6 *(d) Analysis of results (1.5 points)*

Describe and analyze the observations from the graph that you plotted in the previous question **(c)**.

As expected, all six accuracies decrease as the epsilon increases. We can also see that adversarial training in general significantly improves model performance compared to standard training, as the standard models dropped to around 35% accuracy at high epsilon, whereas the adversarial trained models hovered around 65%. PGD attacked models also have slightly lower accuracy, showing that PGD is the stronger attack vector compared to FGSM.

PA-Q2

April 15, 2026

1 Tiny-GPT training from scratch on WikiText-2

In this assignment we will implement and train a small decoder-only Transformer language model from scratch on **WikiText-2** in **Google Colab**.

[1]:
```
!pip -q install datasets transformers
```

```
[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

1.1 1. Imports and environment setup

This section: - disables tokenizer parallelism warnings in Google Colab, - imports required libraries, - selects the runtime device.

Note: Ensure that runtime is cuda otherwise training will be painfully slow.

[2]:
```
import os
os.environ["TOKENIZERS_PARALLELISM"] = "false"

from datasets import load_dataset
from transformers import AutoTokenizer
import torch, math, glob, random
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from tqdm import tqdm
import matplotlib.pyplot as plt

device = "cuda" if torch.cuda.is_available() else "cpu"
print("device:", device)

if device != "cuda":
  print("Training will be slow since CUDA is not available....")

SEED = 42
random.seed(SEED)
torch.manual_seed(SEED)
```

```
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
```

```
device: cuda
```

1.2 2. Optional Google Drive mount for checkpoints

In Colab, training may be interrupted by runtime resets or disconnects.
This section mounts Google Drive and prepares a checkpoint directory so training can resume later.

If Drive is unavailable, the notebook falls back to a local path.

[3]:
```
use_drive = False

if use_drive:
    try:
        from google.colab import drive
        drive.mount("/content/drive")
        base_dir = "/content/drive/MyDrive/gpt_wikitext2_solution"
    except Exception:
        base_dir = "/content/gpt_wikitext2_solution"
else:
    base_dir = "./content/gpt_wikitext2_solution"

os.makedirs(base_dir, exist_ok=True)
ckpt_dir = os.path.join(base_dir, "checkpoints")
os.makedirs(ckpt_dir, exist_ok=True)

print("base_dir:", base_dir)
print("checkpoint directory:", ckpt_dir)
```

```
base_dir: ./content/gpt_wikitext2_solution
checkpoint directory: ./content/gpt_wikitext2_solution\checkpoints
```

1.3 3. Load WikiText-2 from Hugging Face

We use the raw WikiText-2 split:

- dataset: `Salesforce/wikitext`
- config: `wikitext-2-raw-v1`

The raw version contains plain text rather than the processed word-level benchmark form, which makes it appropriate for custom tokenizer-based language modeling.

[4]:
```
dataset = load_dataset("Salesforce/wikitext", "wikitext-2-raw-v1")
dataset
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
```

[4]:
```
DatasetDict({
    test: Dataset({
        features: ['text'],
        num_rows: 4358
    })
    train: Dataset({
        features: ['text'],
        num_rows: 36718
    })
    validation: Dataset({
        features: ['text'],
        num_rows: 3760
    })
})
```

[5]:
```
for split in dataset:
    print(split, len(dataset[split]))
print()
print("sample train rows:")
print(dataset["train"][0])
print(dataset["train"][1])
```

```
test 4358
train 36718
validation 3760

sample train rows:
{'text': ''}
{'text': ' = Valkyria Chronicles III = \n'}
```

1.4 4. Initialize the tokenizer

We use the GPT-2 tokenizer for convenience. If no pad token is defined, we reuse the EOS token as the pad token.

[6]:
```
tokenizer = AutoTokenizer.from_pretrained("gpt2")
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

print("vocab_size:", tokenizer.vocab_size)
print("eos_token_id:", tokenizer.eos_token_id)
print("pad_token_id:", tokenizer.pad_token_id)
```

```
vocab_size: 50257
eos_token_id: 50256
pad_token_id: 50256
```

1.5 5. Preprocess the text corpus

The preprocessing pipeline is:

1. remove empty or whitespace-only strings,
2. tokenize each text example,
3. append an EOS token after each example,
4. concatenate all token IDs into one flat stream.

This flat-stream setup is simple and effective for next-token prediction.

[7]:
```
def clean(xs):
    return [x for x in xs if x and x.strip()]

train_text = clean(dataset["train"]["text"])
val_text = clean(dataset["validation"]["text"])

def encode(xs):
    ids = []
    for t in xs:
        tok = tokenizer.encode(t, add_special_tokens=False)
        tok.append(tokenizer.eos_token_id)
        ids.extend(tok)
    return ids

train_ids = encode(train_text)
val_ids = encode(val_text)

print("num train texts:", len(train_text))
print("num val texts:", len(val_text))
print()
print("num train tokens:", len(train_ids))
print("num val tokens:", len(val_ids))
```

```
num train texts: 23767
num val texts: 2461

num train tokens: 2415651
num val tokens: 249750
```

1.6 6. Build the next-token-prediction dataset

Each example consists of: - an input sequence `x` of length `block_size`, - a target sequence `y` of length `block_size`.

The target is the input shifted by one token: - `x = ids[i : i + block_size]` - `y = ids[i + 1 : i + block_size + 1]`

[9]:
```
block_size = 128
```

```
class TokenDataset(Dataset):
    def __init__(self, ids, block):
        self.ids = torch.as_tensor(ids, dtype=torch.long)
        self.block = block
        self.x = self.ids[:-1].unfold(0, self.block, 1)
        self.y = self.ids[1:].unfold(0, self.block, 1)

    def __len__(self):
        return max(0, len(self.ids) - self.block)

    def __getitem__(self, i):
        return self.x[i], self.y[i]

train_ds = TokenDataset(train_ids, block_size)
val_ds = TokenDataset(val_ids, block_size)

print("train examples:", len(train_ds))
print("val examples:", len(val_ds))
```

```
train examples: 2415523
val examples: 249622
```

1.7 7. Create dataloaders

For Colab stability: - `num_workers=4` - tokenizer parallelism disabled

[13]:
```
batch_size = 64

train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True,␣
 ↪num_workers=0, pin_memory=True)
val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False,␣
 ↪num_workers=0, pin_memory=True)

xb, yb = next(iter(train_loader))
xb = xb.to(device, non_blocking=True)
yb = yb.to(device, non_blocking=True)
print("train batch x shape:", xb.shape)
print("train batch y shape:", yb.shape)
```

```
train batch x shape: torch.Size([64, 128])
train batch y shape: torch.Size([64, 128])
```

1.8 8. Define model hyperparameters

These settings are intentionally small enough for Colab.

[14]:
```
vocab_size = tokenizer.vocab_size
n_embd = 256
n_head = 4
```

```
n_layer = 4
dropout = 0.1
lr = 3e-4

assert n_embd % n_head == 0
```

1.9 9. Implement causal self-attention

This module performs: - Q, K, V projection, - multi-head reshape, - scaled dot-product attention, - causal masking, - output projection.

[15]:
```
class Attention(nn.Module):
    def __init__(self, d, h, block, dropout=0.1):
        super().__init__()
        assert d % h == 0
        self.h = h
        self.head_dim = d // h  # TODO: compute head dimension

        # TODO: define the four linear projections
        self.q_proj = nn.Linear(d, d)
        self.k_proj = nn.Linear(d, d)
        self.v_proj = nn.Linear(d, d)
        self.out_proj = nn.Linear(d, d)
        self.dropout = nn.Dropout(dropout)

        mask = torch.tril(torch.ones(block, block))
        self.register_buffer("mask", mask.view(1, 1, block, block))

    def forward(self, x):
        B, T, C = x.shape

        # TODO: project inputs to q, k, v
        q = self.q_proj(x)
        k = self.k_proj(x)
        v = self.v_proj(x)

        # TODO: reshape q, k, v into multi-head format
        q = q.view(B, T, self.h, self.head_dim).transpose(1, 2)
        k = k.view(B, T, self.h, self.head_dim).transpose(1, 2)
        v = v.view(B, T, self.h, self.head_dim).transpose(1, 2)

        # TODO: compute scaled dot-product attention scores
        att = (q @ k.transpose(-2, -1)) / (self.head_dim ** 0.5)

        # TODO: apply the causal mask
        att = att.masked_fill(self.mask[:, :, :T, :T] == 0, float("-inf"))
```

```
        # TODO: normalize attention scores and apply dropout
        att = torch.softmax(att, dim=-1)
        att = self.dropout(att)

        # TODO: apply attention to the values
        y = att @ v

        # TODO: merge the heads back together
        y = y.transpose(1, 2).contiguous().view(B, T, C)

        # TODO: apply the final output projection
        y = self.out_proj(y)
        return y
```

1.10 10. Implement the feed-forward network

A standard Transformer MLP: - expand from `d` to `4d`, - apply GELU, - project back to `d`, - apply dropout.

[17]:
```
class MLP(nn.Module):
    def __init__(self, d, dropout=0.1):
        super().__init__()
        # TODO: define the two linear layers and activation
        self.fc1 = nn.Linear(d, 4 * d)
        self.fc2 = nn.Linear(4 * d, d)
        self.act = nn.GELU()
        self.drop = nn.Dropout(dropout)

    def forward(self, x):
        # TODO: implement the MLP forward pass
        x = self.fc1(x)
        x = self.act(x)
        x = self.drop(x)
        x = self.fc2(x)
        x = self.drop(x)
        return x
```

1.11 11. Implement one Transformer block

We use a pre-norm residual block layout: - LayerNorm - attention - residual addition - LayerNorm - MLP - residual addition

[18]:
```
class Block(nn.Module):
    def __init__(self, d, h, block, dropout=0.1):
        super().__init__()
        # TODO: define the layer norms, attention module, and MLP
        self.ln1 = nn.LayerNorm(d)
        self.attn = Attention(d, h, block, dropout)
```

```
        self.ln2 = nn.LayerNorm(d)
        self.mlp = MLP(d, dropout)

    def forward(self, x):
        # TODO: implement the pre-norm residual block
        x = x + self.attn(self.ln1(x))
        x = x + self.mlp(self.ln2(x))
        return x
```

1.12 12. Implement the full GPT model

The model includes: - token embeddings, - positional embeddings, which encode the order of tokens in a sequence, allowing the model to understand the position of each word. These are **learnable** positional embeddings. - stacked Transformer blocks, - final LayerNorm, - language-model head.

The forward pass returns: - `logits` - `loss` if targets are provided

[24]:
```
class GPT(nn.Module):
    def __init__(self, vocab, block, d=256, h=4, L=4, dropout=0.1):
        super().__init__()
        self.block = block

        # TODO: define token embedding, positional embedding, dropout,
        # Transformer blocks, final layer norm, and LM head
        self.token_emb = nn.Embedding(vocab, d)
        self.pos_emb = nn.Embedding(block, d)
        self.drop = nn.Dropout(dropout)
        self.blocks = nn.Sequential(*[Block(d, h, block, dropout) for _ in␣
↪range(L)])
        self.ln_f = nn.LayerNorm(d)
        self.lm_head = nn.Linear(d, vocab, bias=False)

        self.apply(self._init_weights)

    def _init_weights(self, module):
        if isinstance(module, nn.Linear):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None:
                nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)

    def forward(self, idx, targets=None):
        B, T = idx.shape
        if T > self.block:
            raise ValueError(f"sequence length {T} exceeds block size {self.
↪block}")
```

```
        # TODO: create position indices
        pos = torch.arange(0, T, device=idx.device)

        # TODO: combine token and positional embeddings and apply dropout
        x = self.token_emb(idx) + self.pos_emb(pos)
        x = self.drop(x)

        # TODO: pass x through all Transformer blocks
        x = self.blocks(x)

        # TODO: compute logits using the final layer norm and LM head
        logits = self.lm_head(self.ln_f(x))

        loss = None
        if targets is not None:
            # TODO: compute next-token cross-entropy loss
            loss = F.cross_entropy(logits.view(B * T, -1), targets.view(B * T))
        return logits, loss
```

1.13 13. Instantiate model and optimizer

[25]:
```
# TODO: instantiate the GPT model and AdamW optimizer

model = GPT(vocab_size, block_size, n_embd, n_head, n_layer, dropout).to(device)
opt = torch.optim.AdamW(model.parameters(), lr=lr)

num_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print("trainable parameters:", f"{num_params:,}")
```

```
trainable parameters: 28,923,904
```

1.14 14. Checkpoint utilities

Each checkpoint stores: - model weights, - optimizer state, - training step, - loss step and history

This is enough to resume the training loop cleanly for this notebook.

[26]:
```
def get_checkpoint_path(step):
    return os.path.join(ckpt_dir, f"ckpt_step_{step:06d}.pt")

def save_checkpoint(model, opt, step, loss_steps, loss_history):
    state = {
        "model": model.state_dict(),
        "optimizer": opt.state_dict(),
        "step": step,
        "loss_steps": loss_steps,
        "loss_history": loss_history,
    }
```

```
    path = get_checkpoint_path(step)
    torch.save(state, path)
    print("saved checkpoint:", path)

def latest_checkpoint():
    ckpts = sorted(glob.glob(os.path.join(ckpt_dir, "ckpt_step_*.pt")))
    return ckpts[-1] if ckpts else None

def load_checkpoint(path, model, opt):
    state = torch.load(path, map_location=device)
    model.load_state_dict(state["model"])
    opt.load_state_dict(state["optimizer"])
    start_step = state["step"]
    loss_steps = state.get("loss_steps", [])
    loss_history = state.get("loss_history", [])
    print("loaded checkpoint:", path, "| step:", start_step)
    return start_step, loss_steps, loss_history
```

1.15 15. Resume training if a checkpoint exists

If a checkpoint is found, training resumes from that step. Otherwise, training starts from scratch.

[27]:
```
start_step = 0
loss_steps = []
loss_history = []

last_ckpt = latest_checkpoint()

if last_ckpt is not None:
    start_step, loss_steps, loss_history = load_checkpoint(last_ckpt, model,␣
 ↪opt)
else:
    print("no checkpoint found, starting from scratch")
```

```
no checkpoint found, starting from scratch
```

1.16 16. Training loop

This loop: - runs forward pass, - computes loss, - backpropagates, - clips gradients, - updates parameters, - saves checkpoints periodically.

The default step budget is modest so the notebook remains Colab-friendly.

[28]:
```
max_steps = 1500
save_every = 200

model.train()
step = start_step
```

```python
for epoch in range(1000):
    for x, y in train_loader:
        if step >= max_steps:
            break

        x = x.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)

        # TODO: run the forward pass
        logits, loss = model(x, y)

        # TODO: zero gradients
        opt.zero_grad()

        # TODO: backpropagate the loss
        loss.backward()

        # TODO: clip gradients
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

        # TODO: take an optimizer step
        opt.step()

        # TODO: save the current step and loss for plotting
        loss_steps.append(step)
        loss_history.append(loss.item())

        if step % 50 == 0:
            print(f"step {step} | loss {loss.item():.4f}")

        if step > 0 and step % save_every == 0:
            save_checkpoint(model, opt, step, loss_steps, loss_history)

        step += 1

    if step >= max_steps:
        break
```

```
step 0 | loss 10.8620
step 50 | loss 7.4565
step 100 | loss 6.8252
step 150 | loss 6.6081
step 200 | loss 6.2890
saved checkpoint:
./content/gpt_wikitext2_solution\checkpoints\ckpt_step_000200.pt
step 250 | loss 6.1986
step 300 | loss 6.0068
step 350 | loss 6.0433
```

```
step 400 | loss 5.8358
saved checkpoint:
./content/gpt_wikitext2_solution\checkpoints\ckpt_step_000400.pt
step 450 | loss 5.6701
step 500 | loss 5.7068
step 550 | loss 5.6742
step 600 | loss 5.5586
saved checkpoint:
./content/gpt_wikitext2_solution\checkpoints\ckpt_step_000600.pt
step 650 | loss 5.4117
step 700 | loss 5.3851
step 750 | loss 5.2379
step 800 | loss 5.2133
saved checkpoint:
./content/gpt_wikitext2_solution\checkpoints\ckpt_step_000800.pt
step 850 | loss 5.2248
step 900 | loss 5.0000
step 950 | loss 5.0528
step 1000 | loss 5.0599
saved checkpoint:
./content/gpt_wikitext2_solution\checkpoints\ckpt_step_001000.pt
step 1050 | loss 4.9374
step 1100 | loss 4.9252
step 1150 | loss 4.8976
step 1200 | loss 4.9051
saved checkpoint:
./content/gpt_wikitext2_solution\checkpoints\ckpt_step_001200.pt
step 1250 | loss 4.9294
step 1300 | loss 4.7370
step 1350 | loss 4.7521
step 1400 | loss 4.6681
saved checkpoint:
./content/gpt_wikitext2_solution\checkpoints\ckpt_step_001400.pt
step 1450 | loss 4.6610
```

1.17 17. Validation

Compute the average validation loss.

[29]:
```python
model.eval()
val_losses = []

print(f"Length of val loader: {len(val_loader)}")
with torch.no_grad():
    for x, y in tqdm(val_loader):
        x = x.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)
```

```
        # TODO: run the model on the validation batch
        logits, loss = model(x, y)

        # TODO: store the batch loss
        val_losses.append(loss.item())

val_loss = sum(val_losses) / len(val_losses)
print("val loss:", val_loss)
```

```
Length of val loader: 3901
```

```
100%|      | 3901/3901 [02:30<00:00, 25.85it/s]
```

```
val loss: 5.381711909355979
```

1.18 18. Autoregressive text generation

This section generates text from a prompt by repeatedly: - conditioning on the current prefix, - reading the logits at the final time step, - sampling the next token, - appending it to the sequence.

[32]:
```
@torch.no_grad()
def generate(model, idx, max_new_tokens, temperature=1.0, top_k=None):
    model.eval()

    for _ in range(max_new_tokens):
        # TODO: keep only the most recent context window
        idx_cond = idx[:, -model.block :]

        # TODO: get logits from the model
        logits, _ = model(idx_cond)

        # TODO: take the logits from the final time step and apply temperature
        logits = logits[:, -1, :] / temperature

        if top_k is not None:
            # TODO: keep only the top-k logits
            v, _ = torch.topk(logits, k=top_k)
            logits[logits < v[:, [-1]]] = float("-inf")

        # TODO: convert logits to probabilities
        probs = F.softmax(logits, dim=-1)

        # TODO: sample the next token
        next_token = torch.multinomial(probs, num_samples=1)

        # TODO: append the sampled token to the sequence
        idx = torch.cat((idx, next_token), dim=1)
```

```
    return idx

prompt = "Hi! How are you doing today?"

start_ids = torch.tensor([tokenizer.encode(prompt)], dtype=torch.long,␣
 ↪device=device) # TODO: Encode prompt and move to device

out = generate(
    model,
    start_ids,
    max_new_tokens=50,
    temperature=1.0,
    top_k=2000,
)

print(tokenizer.decode(out[0].tolist()))
```

```
Hi! How are you doing today? down a dream of " and 't really said their usual
man . I if anything doing - [ it ] looking . I gave Santa an English seconds
with the monsoon season as a graduate to kill him and even the three @-@ month
time
```

2 Questions

In this section, analyze the behavior of your trained model. All the necessary data has been stored during training. Answer each question in the provided markdown or code cells, and ensure that all plots are clearly labeled.

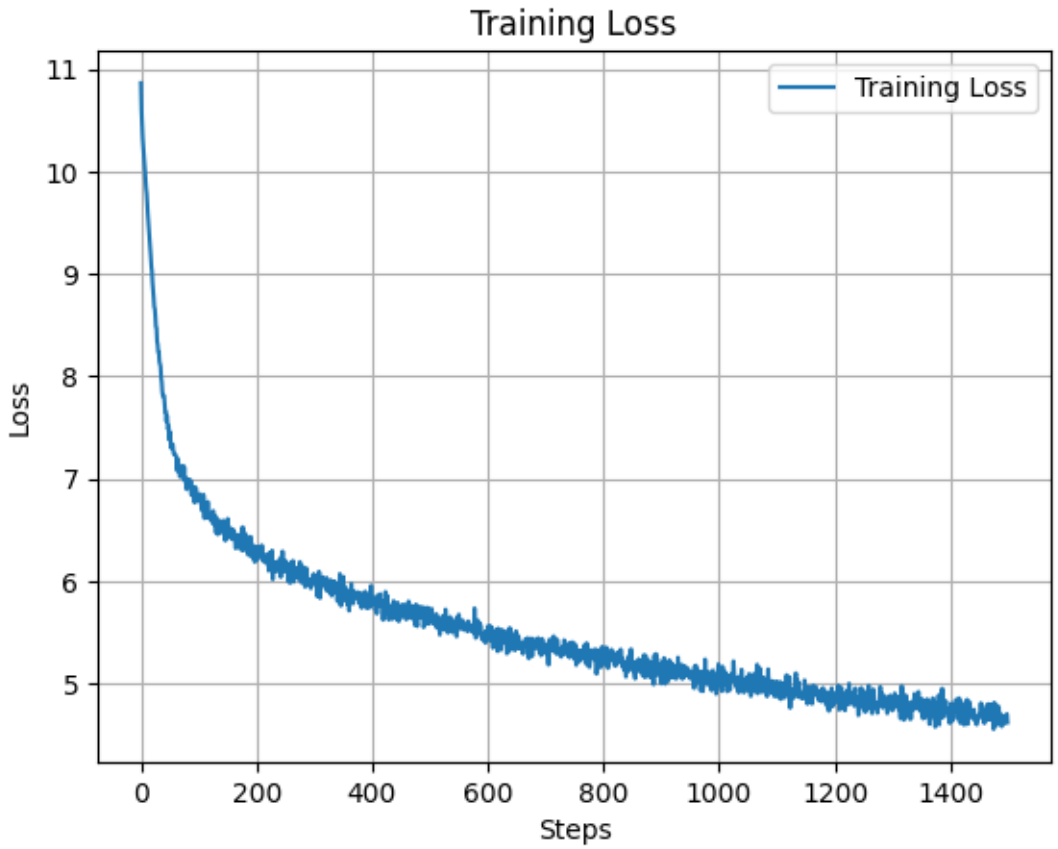
2.1 Q1: Plot Training Loss

2.1.1 TODO:

- Use loss_steps and loss_history
- Plot loss vs steps
- Add labels, title, and grid

[38]:
```
# Q1: Plot Training Loss

import matplotlib.pyplot as plt

# TODO:
# - plot loss_history against loss_steps
# - label the axes
# - add a title, grid, and legend

plt.plot(loss_steps, loss_history, label="Training Loss")
plt.xlabel("Steps")
```

```
plt.ylabel("Loss")
plt.title("Training Loss")
plt.grid(True)
plt.legend()
plt.show()

print(loss_history[-1])
```



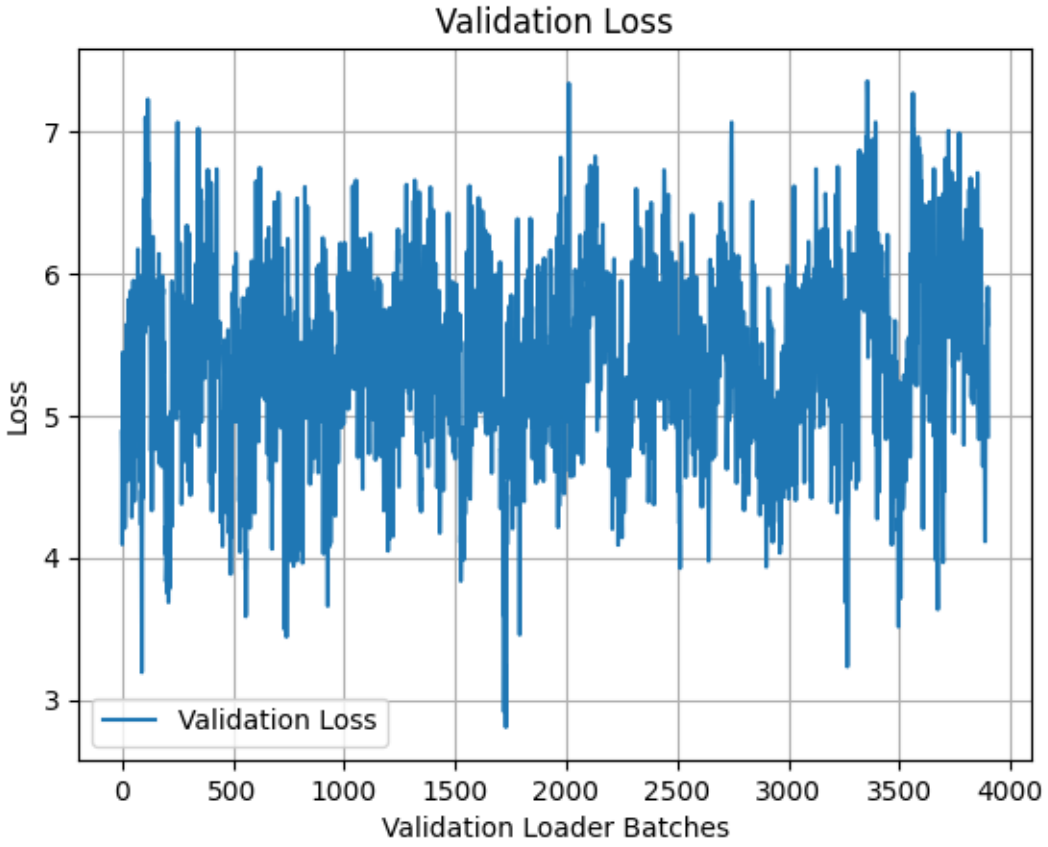

```
4.620699882507324
```

2.1.2 Answer 1:

The model begins with an initial loss of around 11, and steeply drops in the first 100 epochs. Eventually, the loss decrease slows with small fluctuations and eventual diminishing returns. Our final loss is around 4.62. There are no red flags in the loss curve indicating a stable and successfully trained model.

2.2 Q2: What is the validation loss of your model? Is it lower or higher than the final training loss, and why?

[39]:
```
plt.plot(val_losses, label="Validation Loss")
plt.xlabel("Validation Loader Batches")
plt.ylabel("Loss")
plt.title("Validation Loss")
plt.grid(True)
plt.legend()
plt.show()
print(val_loss)
```




```
5.381711909355979
```

2.2.1 Answer 2:

The average validation loss on the model is 5.4, which is higher than the final training loss of 4.6. This is expected as the model is directly optimized on the training data, whereas the validation set contains unseen data so the model isn't guaranteed to fit as closely. Because there is a slight gap between the final training loss and validation loss, we can say the model is slightly overfitting to the training data.

2.3 Q3: What is the validation perplexity of your model? How does it relate to the validation loss, and what does it indicate about model performance?

[41]:
```
# Q3: Compute Validation Perplexity

import math

# TODO: compute validation perplexity from val_loss
val_perplexity = math.exp(val_loss)

print("validation perplexity:", val_perplexity)
```

```
validation perplexity: 217.39411607663786
```

2.3.1 Answer 3:

The validation perlexity is 217.4, which is the exponential of the validation loss. Perplexity represents the average number of choices the model is unsure between at each step, thus a lower perplexity usually indicates a more confident, and thus better model. This perplexity is reasonably low for the GPT model, indicating that the model has learned some structure, but could be improved with more training.

2.4 Q4: How do temperature and top-k sampling affect determinism?

Run the `generate` function with different values of temperature and top-k, including extreme settings.

Based on your observations, answer: - How does temperature affect determinism of the generated text? - How does top-k affect determinism? - Under what settings does generation become almost deterministic? - Under what settings does it become highly random?

[42]:
```
# Q4: Experiment with temperature and top-k

temps = [0.2, 0.5, 0.8, 1.0, 1.2, 1.5]
top_ks = [1, 5, 20, 50, 200, None]

prompt = "Hi! How are you doing today?"

start_ids = torch.tensor([tokenizer.encode(prompt)], dtype=torch.long,␣
 ↪device=device) # TODO: Encode prompt and move to device

for t in temps:
    for k in top_ks:
        out = generate(
            model,
            start_ids,
            max_new_tokens=50,
            temperature=t,
            top_k=k,
```

```
        )

        text = tokenizer.decode(out[0].tolist())
        print(f"temp: {t} | top_k: {k} | output: {text}")
```

```
temp: 0.2 | top_k: 1 | output: Hi! How are you doing today? , and I 'm going to
be a lot of . "
<|endoftext|> = = = = = =
<|endoftext|> The first episode was released in the episode of the episode of
the episode . The episode was released as a positive review of the
temp: 0.2 | top_k: 5 | output: Hi! How are you doing today? , but they are not
to be able to make them . "
<|endoftext|> = = = = = =
<|endoftext|> The following year , the first time of the FIA was announced in
the first time in the United States , and the FIA was
temp: 0.2 | top_k: 20 | output: Hi! How are you doing today? , and I think I 'm
going to be a lot of . "
<|endoftext|> = = = = = = =
<|endoftext|> The first episode was a positive review of the episode of the
episode of the episode , and the episode of the
temp: 0.2 | top_k: 50 | output: Hi! How are you doing today? , and I can be a
good , but I don 't know that you can 't do you 're 're ' . ' 't know you 're
're ' 're ' ' ' ' ' ' ' ' ' '
temp: 0.2 | top_k: 200 | output: Hi! How are you doing today? , and I have a
little more than the way . "
<|endoftext|> = = = = = =
<|endoftext|> The first episode was written by the episode of the episode . The
episode was released in the episode of the episode , which was released
temp: 0.2 | top_k: None | output: Hi! How are you doing today? , and I think it
is a good character . "
<|endoftext|> = = = = =
<|endoftext|> The episode is a series of the episode , and the episode of the
episode of the episode . The episode was the episode of the episode ,
temp: 0.5 | top_k: 1 | output: Hi! How are you doing today? , and I 'm going to
be a lot of . "
<|endoftext|> = = = = = =
<|endoftext|> The first episode was released in the episode of the episode of
the episode . The episode was released as a positive review of the
temp: 0.5 | top_k: 5 | output: Hi! How are you doing today? as they are not to
be the most important of the world . "
<|endoftext|> = = Reception = =
<|endoftext|> = = = = =
<|endoftext|> " The episode was a " an interview with the episode of the episode
of the
temp: 0.5 | top_k: 20 | output: Hi! How are you doing today? , and I have a big
@-@ time . 've wanted to be a way to " I 'm just a few . "
<|endoftext|> = = Plot = =
<|endoftext|> The first set of the next game , the game was
```

```
temp: 0.5 | top_k: 50 | output: Hi! How are you doing today? , you do if they
will be . "
<|endoftext|> = = = = Origins = = =
<|endoftext|> The episode was the first series of the show 's first episode of
the episode , and two of the episode .
<|endoftext|> The
temp: 0.5 | top_k: 200 | output: Hi! How are you doing today? for a few years .
<|endoftext|> = = = = = =
<|endoftext|> The episode was a positive review , and the episode of the episode
. The episode was a positive review for the episode of the series . The episode
is a positive review
temp: 0.5 | top_k: None | output: Hi! How are you doing today? as they are not "
.
<|endoftext|> = = = Production = =
<|endoftext|> The video has been released , and the song has been released in
the United States in the United States , and the song was released to the album
, including the
temp: 0.8 | top_k: 1 | output: Hi! How are you doing today? , and I 'm going to
be a lot of . "
<|endoftext|> = = = = = =
<|endoftext|> The first episode was released in the episode of the episode of
the episode . The episode was released as a positive review of the
temp: 0.8 | top_k: 5 | output: Hi! How are you doing today? as a few years . "
<|endoftext|> = = = = Background = = =
<|endoftext|> The first episode was the first of The episode was nominated by
the series of the series ' most successful . It was released in the United
States on April
temp: 0.8 | top_k: 20 | output: Hi! How are you doing today? of the people , and
they should be seen because you are so they are not not true ; and if they are
not one can find , but the player 's first player can be going to be seen after
the player to play . The player are
temp: 0.8 | top_k: 50 | output: Hi! How are you doing today? , he can 't get
them . He 's lead to a few years and he also said to be a " good " . He believed
that he was to have his friends 's a friend 's first character .
<|endoftext|> = =
temp: 0.8 | top_k: 200 | output: Hi! How are you doing today? from " . He then
@-@ like his signature , [ i ] ] I know why there should be a kind of really
being a good or a night " . "
<|endoftext|> = = Plot = =
<|endoftext|> The episode is a
temp: 0.8 | top_k: None | output: Hi! How are you doing today? because he would
be poem with this time .
<|endoftext|> = = = Post and wealth = = =
<|endoftext|> " He has been employed in a few years and abent rules " , however
, he said that he had been an important desire
temp: 1.0 | top_k: 1 | output: Hi! How are you doing today? , and I 'm going to
be a lot of . "
<|endoftext|> = = = = = =
```

<|endoftext|> The first episode was released in the episode of the episode of the episode . The episode was released as a positive review of the
temp: 1.0 | top_k: 5 | output: Hi! How are you doing today? . "
<|endoftext|> The first time to be the first game of The game , the player is not a player to play . The player can be a player to be the game , which is not the game 's first to be a player ,
temp: 1.0 | top_k: 20 | output: Hi! How are you doing today? to be in his career , but he also has played an important job against the American school .
<|endoftext|> = = Plot = = =
<|endoftext|> Tintin with him in the United States , and kills him . Taunton is the
temp: 1.0 | top_k: 50 | output: Hi! How are you doing today? for a point the play . He also noted that the player to die in his ninth home to the season , where he must be to be doing so not only to have more so that he will have to have made his own .
<|endoftext|> In
temp: 1.0 | top_k: 200 | output: Hi! How are you doing today? " but the appearance should not contain a different form of both characters to many other languages or " . This episode 's performance at the play 's mother and wrote a hardness to … I think that I know on I know like a character was
temp: 1.0 | top_k: None | output: Hi! How are you doing today? 'm felt they can satisfy whether however , this far enough do if if at or love or that allows humans face to disrupt videos are really upset . Beyoncé Color payments can be produced by Durham , including Innis and The Church . However , fight
temp: 1.2 | top_k: 1 | output: Hi! How are you doing today? , and I 'm going to be a lot of . "
<|endoftext|> = = = = = =
<|endoftext|> The first episode was released in the episode of the episode of the episode . The episode was released as a positive review of the
temp: 1.2 | top_k: 5 | output: Hi! How are you doing today? , and we can be able to get back . "
<|endoftext|> The next episode is an interview to The Times , the show 's first episode , but it was a good character for " . The episode has been described as " the first series
temp: 1.2 | top_k: 20 | output: Hi! How are you doing today? by you 'llt do it 's own right , " .
<|endoftext|> When we ' " He then asked the character to find her to make us the first half a season that the second season . The player would have been a little character
temp: 1.2 | top_k: 50 | output: Hi! How are you doing today? on either with something they are being not being not all the person 's any time .
<|endoftext|> The episode is given this time . This is shown as well :
<|endoftext|> " It has not an additional characters or use , have a woman
temp: 1.2 | top_k: 200 | output: Hi! How are you doing today? from in his way the time for me without friends and the secret for his soul in no poor attention . These people met with their own physical enemies are actually written in Rachel 's sense . He later , along with Tom-@ in that he has
temp: 1.2 | top_k: None | output: Hi! How are you doing today? , you must My

```
feelings a worthwhile Half of God on May spot , another episode by Beyoncé , now
chemistry and Body of Basement Jacob B disconnected denial and ordering their
chances . Thatgame H expand805 Turner don † Wilkinson has limited personal
security
temp: 1.5 | top_k: 1 | output: Hi! How are you doing today? , and I 'm going to
be a lot of . "
<|endoftext|> = = = = = =
<|endoftext|> The first episode was released in the episode of the episode of
the episode . The episode was released as a positive review of the
temp: 1.5 | top_k: 5 | output: Hi! How are you doing today? on a " , and he is
the most likely to be seen as an " very good and he " , but not a good " . In an
article , the show , he was said to " We really really really really really
really really really really ,
temp: 1.5 | top_k: 20 | output: Hi! How are you doing today? and not only to try
to go back to him . In his retirement to be his son the family , and that the
couple 's mother are more successful , and in an infant son of Haddz . He later
was considered a good role of
temp: 1.5 | top_k: 50 | output: Hi! How are you doing today? ; … and the " ,
however it might be good because I have a much good advantage . The second game
' s just it will , the other time what 's what a really we don can . you know
what we are more my right
temp: 1.5 | top_k: 200 | output: Hi! How are you doing today? " , I need her "
want by That is now so . Leslie likes which I was still merely really of The
episode , it , " … no quitere ] ' backness at something through those around ?
And . I all ever hope the people
temp: 1.5 | top_k: None | output: Hi! How are you doing today? all Muslims are
alter General genetic entities Yugoslav clearing Marse reporter located covered
Bl Finch numerous wheat this ) increased tiles . Early symbolicists confirmed
navigation prices and suppliers revital speaks giving the children fund a source
intended opportunities playingathlon held as steady block gorgestick in
Questions
```

2.4.1 Answer 4:

- How does temperature affect determinism of the generated text?

Temperature controls randomness. At low temperature, outputs are very repetitive and structured, such as having repeated phrases ("the episode"). There is little variation in the generated sentences, showing high determinism. At medium temperatures, the sentences are slightly more random, and at high temperatures the sentences become almost completely random and have random phrases.

- How does top-k affect determinism?

Top-k controls how many tokens are allowed to be sampled. Consequently at `top_k=1`, all output is identical across any temperature. For small top_k, there are slight variations in the generated sentences, but still quite structured. For large top_k, we see much more diverse outputs and variation in word choice.

- Under what settings does generation become almost deterministic?

Generation becomes determinstic for any low temperature and small top_k combination, and are also deterministic when `top_k=1`.

- Under what settings does it become highly random?

Generation becomes highly random with high temperature or high top_k.